



Betriebliche Projektarbeit

Fachinformatiker für Anwendungsentwicklung

Abschlussprüfung Winter 2025/26

Entwicklung eines webbasierten Skill Management Systems zur effizienten projektbezogenen Ressourceneinsatzplanung

Prüfling

Marvin Eschenbach
Hauptstraße 10
97653 Bischofsheim i. d. Rhön

Prüflingsnummer

20120

Ausbildungsbetrieb

EDAG Engineering GmbH
Reesbergstraße 1
36039 Fulda

Betrieblicher Betreuer

Christian Henning

Abgabedatum: 01.12.2025

Zusammenfassung

Ausgangssituation

Die EDAG Engineering GmbH, ein international tätiger Engineering-Dienstleister mit Fokus auf Automotive und Mobility, benötigte ein effizientes System zur projektbezogenen Ressourcenplanung in der Abteilung Mobility IT (ca. 100 Mitarbeiter). Die bisherige manuelle Mitarbeitersuche über Excel-Listen und E-Mail-Anfragen war zeitintensiv (durchschnittlich 2,5 Stunden pro Projektbesetzung) und fehleranfällig. Projektmanager verfügten über keine zentrale Übersicht zu Mitarbeiterkompetenzen und Projekterfahrungen[1].

Projektziel

Entwicklung eines webbasierten Skill-Management-Systems zur effizienten, datenbasierten Personalauswahl mit umfassender Kompetenzverwaltung, Such- und Filterfunktionen sowie rollenbasiertem Rechtemanagement für drei Benutzergruppen (Mitarbeiter, Manager, Administrator). Das System sollte die Suchzeit auf unter 15 Minuten reduzieren und eine jährliche Kostenersparnis von ca. 8.500 € ermöglichen (siehe Kapitel 2.4.3).

Technische Umsetzung

Das System wurde in 80 Stunden (1. Oktober bis 1. Dezember 2025) mit modernem Technology-Stack realisiert: Next.js 16 mit React 19 für das Frontend[2, 3], Spring Boot 3.5.7 mit Hexagonaler Architektur für das Backend[4], PostgreSQL 15 als relationale Datenbank[5], Keycloak 23 für Open Authorization (OAuth)2/OpenID Connect (OIDC)-Authentifizierung[6] sowie Kubernetes-Deployment[7] mit Jenkins Continuous Integration/Continuous Deployment (CI/CD)-Pipeline und SonarQube-Qualitätssicherung[8, 9].

Funktionsumfang

Das System bietet personalisierte Dashboards mit grafischer Skill-Visualisierung, umfassende Profilverwaltung (Skills mit Bewertung 0–100, Verfügbarkeit, Projekterfahrung), leistungsfähige Mitarbeitersuche nach Skills, Standort, Verfügbarkeit und Erfahrung, vollständiges Projektmanagement für Manager (Projektanlage, Team-Zusammenstellung) sowie Stammdaten- und Rechteverwaltung für Administratoren. Die Benutzeroberfläche erfüllt moderne Usability-Standards mit Antwortzeiten unter 2 Sekunden[10]. Insgesamt wurden 15 Use Cases (siehe Anhang, Tabelle A.9) implementiert und erfolgreich validiert.

Projektergebnis

Die Implementierung erfüllt alle funktionalen und nicht-funktionalen Anforderungen vollständig. Das skalierbare System mit Clean Architecture bietet eine solide Grundlage für den unternehmensweiten Einsatz und zukünftige Erweiterungen wie KI-gestützte Skill-Empfehlungen oder Integration in bestehende HR-Systeme.

Persönliche Erklärung

Ich versichere durch meine Unterschrift, dass ich die vorliegende Projektdokumentation selbstständig und ohne fremde Hilfe angefertigt habe.

Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder nicht veröffentlichten Schriften entnommen wurden, sind als solche kenntlich gemacht.

Die Arbeit hat in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegen.

Ich bin damit einverstanden, dass die Projektdokumentation in der Bibliothek der IHK zur Einsichtnahme ausgelegt wird.

Wegfurt, 01.12.2025

Ort, Datum



Unterschrift Prüfling

Das Projekt wurde

☒ abgenommen

☐ nicht abgenommen

Petersberg, 01.12.2025

Ort, Datum



Unterschrift betrieblicher Betreuer

Inhaltsverzeichnis

Zusammenfassung	i
Persönliche Erklärung	ii
Abkürzungsverzeichnis	x
Glossar	xi
1 Einleitung	1
1.1 Projektumfeld	1
1.1.1 Unternehmen	1
1.1.2 Abteilung	1
1.2 Ist-Analyse	1
1.2.1 Probleme des aktuellen Prozesses	1
1.2.2 Wirtschaftliche Auswirkungen	2
1.3 Zielsetzung	2
1.3.1 Funktionale Anforderungen	2
1.3.2 Nicht-funktionale Anforderungen	3
1.3.3 Use-Case-Übersicht	3
1.4 Projektnutzen	4
1.5 Projektabgrenzung	5
2 Projektplanung	6
2.1 Projektphasenmodell	6
2.1.1 Phasenübersicht	6
2.2 MVP-Definition und Priorisierung	7
2.2.1 MVP-Kernfunktionen	7
2.2.2 Abgrenzung: Nicht im MVP enthalten	7
2.2.3 Gantt-Diagramm	7
2.3 Ressourcenplanung	8
2.4 Wirtschaftlichkeitsbetrachtung	8
2.4.1 Kostenplanung	8
2.4.2 Make-or-Buy-Entscheidung	9
2.4.3 Nutzenanalyse	9
2.4.4 Amortisationsrechnung	9

3	 Projektdurchführung	11
3.1	Entwicklungsumgebung	11
3.2	Datenbankdesign und -implementierung	11
3.2.1	Datenmodell	11
3.2.2	Datenbank-Migration mit Flyway	12
3.3	Backend-Implementierung	12
3.3.1	Architektur	12
3.3.2	Schichtenaufteilung	13
3.3.3	Zentrale API-Endpunkte	13
3.3.4	Authentifizierung und Autorisierung	14
3.4	Frontend-Implementierung	14
3.4.1	Technologie-Stack und State Management	14
3.4.2	Komponentenstruktur	15
3.4.3	Kernfunktionen	15
3.4.4	State Management	16
3.5	Systemintegration	16
3.5.1	Keycloak-Anbindung und User-Synchronisation	16
3.5.2	Rollenanfrage-Workflow (Self-Service)	16
3.6	Kubernetes-Deployment	17
3.7	CI/CD-Pipeline	17
3.8	Qualitätssicherung	18
4	 Projektergebnisse	19
4.1	Funktionale Systemvalidierung	19
4.1.1	Testabdeckung	19
4.1.2	Code-Qualitätsmetriken	19
4.1.3	Performance-Ergebnisse	19
4.2	Soll-Ist-Vergleich	20
4.2.1	Zeitliche Zielerreichung	20
4.2.2	Funktionale Zielerreichung	20
4.2.3	Wirtschaftliche Zielerreichung	21
4.3	Systemdemonstration	21
4.4	Abweichungen und Anpassungen	21
5	 Fazit	22
5.1	Projektzusammenfassung	22
5.1.1	Zentrale Ergebnisse	22
5.2	Zielerreichung	22
5.2.1	Wirtschaftliche Ziele	22
5.2.2	Qualitätsziele	23
5.3	Reflexion und Lernerfahrungen	23
5.4	Ausblick und Weiterentwicklung	23
5.5	Schlusswort	24

A	Anhang	25
A.1	Projektplanung & Wirtschaftlichkeit	25
A.1.1	Detaillierte Zeitplanung	25
A.1.2	Technologie-Stack	26
A.1.3	Wirtschaftlichkeitsberechnung Details	26
A.1.3.1	IST-Prozess (manuell)	26
A.1.3.2	SOLL-Prozess (automatisiert)	27
A.1.4	Make-or-Buy-Vergleich	27
A.2	Architektur & Implementierung	27
A.2.1	Repository-Struktur	27
A.2.2	Code-Beispiele	30
A.2.2.1	Backend: Employee Discovery Request Flow	30
A.2.2.2	Frontend: Projektdetail-Feature	36
A.2.2.3	Redux Toolkit Query API Slice	38
A.2.2.4	Webhook-basierte User-Synchronisation	39
A.2.2.5	Rollenbasierte Autorisierung	40
A.3	Diagramme	41
A.3.1	Sequenzdiagramme	41
A.3.1.1	OAuth2 Authentifizierung und Autorisierung	41
A.3.1.2	Mitarbeitersuche mit dynamischer Filterung	42
A.3.1.3	Skill-Verwaltung (CRUD-Operationen)	44
A.3.2	Aktivitätsdiagramme	46
A.3.2.1	Projekterstellung mit Mitarbeiterauswahl	46
A.3.2.2	Mitarbeitersuche mit Multi-Filter	47
A.3.3	Klassendiagramm	48
A.3.3.1	Domain Model	48
A.3.4	Zustandsdiagramm	50
A.3.4.1	Rollenanfrage-Workflow	50
A.4	Qualitätssicherung & CI/CD	50
A.4.1	Code-Qualitätsmetriken	50
A.4.2	CI/CD und Quality Assurance	51
A.4.2.1	Commit-Historie und Conventional Commits	51
A.4.2.2	SonarQube Code-Analyse	51
A.4.2.3	Jenkins CI/CD Pipeline	52
A.4.2.4	Jenkinsfile Deklaration	53
A.4.2.5	Lighthouse Performance Audit	55
A.5	Deployment & Konfiguration	56
A.5.1	ER-Datenmodell	56
A.5.1.1	Datenbankmodell	56
A.5.2	Datenbank-Migrationen	57
A.5.2.1	Migrations-Übersicht	58
A.5.3	Kubernetes Ingress-Konfiguration	59

A.6	Dokumentation	60
A.6.1	UI-Screenshots	60
A.6.1.1	Authentifizierung und Onboarding	60
A.6.1.2	Dashboard und Home	62
A.6.1.3	Profilverwaltung	63
A.6.1.4	Mitarbeiter- und Projektsuche	64
A.6.1.5	Administrationsoberfläche	67
A.6.1.6	UI-Mockup (Planungsphase)	68
A.6.2	Use Cases	68
A.6.2.1	Übersicht der 15 implementierten Use Cases	68
A.6.3	Benutzer- und Entwicklerdokumentation	69
A.6.4	API-Dokumentation	70
A.6.4.1	Vollständige API-Endpunkt-Übersicht	71
Literaturverzeichnis		73

Abbildungsverzeichnis

1.1	Use-Case-Diagramm des Skill Management Systems	4
2.1	Gantt-Diagramm des Projektzeitplans	8
2.2	Amortisationsanalyse über 12 Monate	10
3.1	Hexagonale Architektur des Backend-Systems	13
3.2	Kubernetes-Architektur: 3 Namespaces mit Services und Ingress-Routing	17
A.1	OAuth2 Token Introspection Flow mit rollenbasierter Autorisierung	41
A.2	Mitarbeitersuche: Request-Flow mit JPA Specification Pattern	43
A.3	Skill-Verwaltung: CRUD-Workflow mit RTK Query Cache-Management	45
A.4	Aktivitätsdiagramm: Projekterstellung mit Mitarbeiterauswahl	47
A.5	Aktivitätsdiagramm: Mitarbeitersuche mit dynamischen Filtern	48
A.6	Skill Management System: Domain Model - Kernentitäten	49
A.7	Zustandsdiagramm: Rollen-anfrage-Workflow (Self-Service Pattern)	50
A.8	Commit-Historie mit Conventional Commits und Commit Lint	51
A.9	SonarQube Analyse – Backend (SMS Backend, Branch: main)	51
A.10	SonarQube Analyse – Frontend (SMS Frontend, Branch: main)	52
A.11	Jenkins Declarative Pipeline – Build #7 (Trigger Deploy Stage)	52
A.12	Google Lighthouse Audit – Welcome Screen (Performance: 100, Best Practices: 81)	55
A.13	Entity-Relationship-Diagramm nach UML-Standard (Kernentitäten)	56
A.14	Datenbank-Schema: Vollständige Tabellenstruktur mit 15 Tabellen	57
A.15	Login-Seite mit Keycloak OAuth2/OIDC-Integration	60
A.16	Keycloak-Authentifizierungsmaske	60
A.17	Login-Erfolgsmeldung und Weiterleitung	61
A.18	Welcome-Seite: Erste Schritte nach Registrierung	61
A.19	Welcome-Seite: Profilkonfiguration	61
A.20	Welcome-Seite: Skill-Auswahl	62
A.21	Dashboard-Übersicht mit personalisierten Kennzahlen	62
A.22	Dashboard mit Skill-Entwicklungstrends	63
A.23	Profil-Bearbeitung: Persönliche Daten und Skills	63
A.24	Profil-Ansicht: Skill-Matrix mit Proficiency-Scores	64
A.25	Mitarbeitersuche mit Skill-basierten Filtern	64

A.26 Projektübersicht mit Status-Filter	65
A.27 Projekt-Detailansicht mit Team-Zusammensetzung	65
A.28 Projekt-Erstellung: Formular für neue Projekte	66
A.29 Projekt-Team-Management: Mitglieder hinzufügen	66
A.30 Admin-Dashboard: Systemstatistiken und Benutzeraktivitäten	67
A.31 Admin-Panel: Rollenanfragen-Verwaltung	67
A.32 Initiales UI-Mockup aus der Planungsphase	68
A.33 Benutzer- und Entwicklerdokumentation: Deckblatt der Benutzerdokumentation	69
A.34 Benutzer- und Entwicklerdokumentation: Auszug aus der funktionalen Übersicht (Kapitel 3)	69
A.35 Benutzer- und Entwicklerdokumentation: Auszug aus der Architekturübersicht (Kapitel 4 - Bausteine)	70
A.36 API-Dokumentation: Übersicht der verfügbaren Endpunkte gruppiert nach Funktionsbereichen	71

Tabellenverzeichnis

2.1	Phasenmodell mit Zeitverteilung	6
2.2	MVP-Feature-Priorisierung nach MoSCoW-Methode[23]	7
2.3	Detaillierte Kostenplanung	9
4.1	Soll-Ist-Vergleich der Projektphasen	20
4.2	Funktionale Anforderungen – Zielerreichung	20
A.1	Detaillierte Zeitplanung des Projekts	25
A.2	Übersicht der eingesetzten Technologien	26
A.3	IST-Prozess bei 50 Projektbesetzungen pro Jahr	26
A.4	SOLL-Prozess mit Skill-Management-System	27
A.5	Make-or-Buy-Vergleich	27
A.6	SonarQube Code-Qualitätsmetriken (Stand: Build #15)	50
A.7	Vollständige Übersicht aller Flyway-Datenbank-Migrationen	58
A.8	Traefik Ingress-Konfiguration für alle Environments	59
A.9	Übersicht der 15 implementierten Use Cases	68
A.10	Vollständige REST-API-Endpunkt-Übersicht des Skill Management Systems	72

Abkürzungsverzeichnis

API Application Programming Interface.

CI/CD Continuous Integration/Continuous Deployment.

DTO Data Transfer Object.

IDE Integrated Development Environment.

IdP Identity Provider.

JWT JSON Web Token.

MVP Minimum Viable Product.

OAuth Open Authorization.

OIDC OpenID Connect.

ORM Object-Relational Mapping.

REST Representational State Transfer.

ROI Return on Investment.

SSO Single Sign-On.

UML Unified Modeling Language.

Glossar

Backend Serverseitige Komponente einer Anwendung, die Geschäftslogik, Datenbankzugriffe und API-Endpunkte bereitstellt.

Clean Architecture Softwarearchitekturmuster, das eine klare Trennung von Geschäftslogik und technischen Details durch Schichtenbildung erreicht.

DevOps Kunstwort aus Development und Operations; beschreibt die Kombination von Softwareentwicklung und IT-Betrieb zur Verbesserung der Zusammenarbeit und Automatisierung.

Frontend Clientseitige Komponente einer Anwendung, die die Benutzeroberfläche und Benutzerinteraktion bereitstellt.

Kanban Agile Methode zur Prozesssteuerung, die Arbeit visualisiert und den Arbeitsfluss durch Work-in-Progress-Limits optimiert.

Quality Gate Definierte Qualitätskriterien in der Softwareentwicklung, die erfüllt sein müssen, bevor Code in die nächste Phase übergehen darf.

Scope Creep Unkontrollierte Erweiterung des Projektumfangs während der Entwicklung durch ungeplante Anforderungen, die zu Zeit- und Kostenüberschreitungen führen kann.

Scrum Agiles Framework für Projektmanagement, das in kurzen Iterationen (Sprints) arbeitet und auf selbstorganisierten Teams basiert.

Skill Fähigkeit oder Kompetenz eines Mitarbeiters, z. B. Programmiersprache, Technologie oder Methodik.

Three-Tier-Architektur Dreischichtige Softwarearchitektur bestehend aus Präsentationsschicht (Frontend), Anwendungsschicht (Backend/Geschäftslogik) und Datenhaltungsschicht (Datenbank).

WCAG Web Content Accessibility Guidelines – Richtlinien für barrierefreie Webinhalte des W3C.

Kapitel 1

Einleitung

1.1 Projektumfeld

1.1.1 Unternehmen

Die EDAG Engineering GmbH ist ein international tätiger Engineering-Dienstleister mit Hauptsitz in Wiesbaden, Deutschland. Als einer der weltweit führenden unabhängigen Engineering-Partner entwickelt EDAG innovative Lösungen für die Automobilindustrie und weitere Mobilitätsbranchen[11]. Das 1969 gegründete Unternehmen beschäftigt mehrere tausend Mitarbeiter an über 80 Standorten weltweit und erwirtschaftet einen Jahresumsatz im hohen dreistelligen Millionen-Euro-Bereich[12].

1.1.2 Abteilung

Das Projekt wird in der Abteilung *Mobility IT* durchgeführt, die innovative Softwarelösungen für Smart City und Automotive-Projekte entwickelt. Das interdisziplinäre Team besteht aus knapp 100 Mitarbeitern mit unterschiedlichen Kompetenzen in den Bereichen Frontend- und Backend-Entwicklung, DevOps, Cloud-Architektur und Projektmanagement. Die Abteilung arbeitet mit modernen Cloud-Native-Technologien wie Docker[13] und Kubernetes[7], agilen Methoden (Scrum, Kanban) und DevOps-Praktiken[14, 15]. Das im Rahmen dieser Projektarbeit entwickelte Skill-Management-System ist als internes Tool konzipiert und soll die Ressourcenplanung innerhalb der Abteilung optimieren.

1.2 Ist-Analyse

Die Personalauswahl für Kundenprojekte erfolgt derzeit durch manuelle, dezentrale Prozesse ohne zentrale Kompetenzübersicht. Projektmanager nutzen veraltete Excel-Tabellen und führen zeitaufwendige E-Mail-Anfragen bei Teamleitern durch, um geeignete Mitarbeiter zu identifizieren.

1.2.1 Probleme des aktuellen Prozesses

- Fehlende zentrale, aktuelle Übersicht über Mitarbeiterkompetenzen und deren Ausprägung

- Zeitaufwendige Suche nach geeigneten Mitarbeitern (durchschnittlich 2,5 Stunden pro Projektbesetzung[1])
- Keine systematische, nachvollziehbare Dokumentation von Skills und Projekterfahrungen
- Erhöhtes Risiko von Fehlbesetzungen durch unvollständige oder veraltete Informationen
- Ineffiziente Ressourcennutzung und potenzielle Überlastung einzelner Mitarbeiter
- Mangelnde Transparenz über verfügbare Kompetenzen behindert strategische Personalentwicklung

1.2.2 Wirtschaftliche Auswirkungen

Bei 50 Projektbesetzungen jährlich entstehen Opportunitätskosten von 9.375 € (125 Stunden $\times$ 75 €/h)[1, 16]. Die detaillierte Wirtschaftlichkeitsbetrachtung erfolgt in Kapitel 2.4.

1.3 Zielsetzung

Das Ziel ist die Entwicklung eines webbasierten Skill-Management-Systems als Minimum Viable Product (MVP), das eine effiziente, datenbasierte Personalauswahl ermöglicht und die Transparenz über Mitarbeiterkompetenzen erhöht. Der MVP-Ansatz fokussiert auf die Kernfunktionalitäten, die den größten Mehrwert für die Abteilung liefern, und ermöglicht eine schnelle Inbetriebnahme mit späteren Erweiterungsmöglichkeiten.

1.3.1 Funktionale Anforderungen

Das MVP unterstützt drei Benutzerrollen mit unterschiedlichen Berechtigungen:

Mitarbeiter (Employee):

- Personalisiertes Dashboard mit Übersicht über Skill-Entwicklung, zugewiesene Projekte und relevante Kennzahlen
- Umfassende Profilverwaltung: Skills mit Selbsteinschätzung (Skala 0–100), persönliche Beschreibung, Berufserfahrung und Verfügbarkeit
- Suchfunktion für Mitarbeiter
- Übersicht über eigene Projektzuordnungen und kürzliche Aktivitäten im System

Projektleiter (Manager):

- Alle Funktionen der Mitarbeiter-Rolle
- Projektmanagement: Projekte anlegen, bearbeiten und Teams zusammenstellen
- Projektsuche mit Filtern nach Skills, Mitarbeiterzuordnungen und Projektstatus

Administrator (Admin):

- Alle Funktionen der Mitarbeiter-Rolle
- Benutzerverwaltung: Rollenzuweisung (Admin, Manager, Mitarbeiter), Bearbeitung von Rollen Anfragen
- Stammdatenverwaltung: Pflege von Skills, Skill-Kategorien, Standorten und Positionen

1.3.2 Nicht-funktionale Anforderungen

- **Usability:** Intuitive Benutzeroberfläche ohne Schulungsaufwand, konform zu Usability-Standards[17]
- **Performance:** Antwortzeiten unter 2 Sekunden für Suchanfragen, maximale Seitenladezeit von 3 Sekunden[18]
- **Sicherheit:** Sichere Authentifizierung über OAuth 2.0 und OIDC mit Keycloak[6], rollenbasierte Zugriffskontrolle[19]
- **Skalierbarkeit:** Architektur für unternehmensweiten Einsatz mit mehreren hundert Benutzern ausgelegt mittels Kubernetes[7]
- **Wartbarkeit:** Modulare Architektur nach Clean Architecture-Prinzipien, Code-Qualitätssicherung[20]
- **Integration:** Automatische Synchronisation von Benutzerdaten aus Keycloak[6], Api-First-Ansatz für zukünftige Integrationen[21]

1.3.3 Use-Case-Übersicht

Abbildung 1.1 zeigt die Hauptfunktionen des Systems und deren Zuordnung zu den drei Benutzerrollen gemäß Unified Modeling Language (UML)-Standard[22].

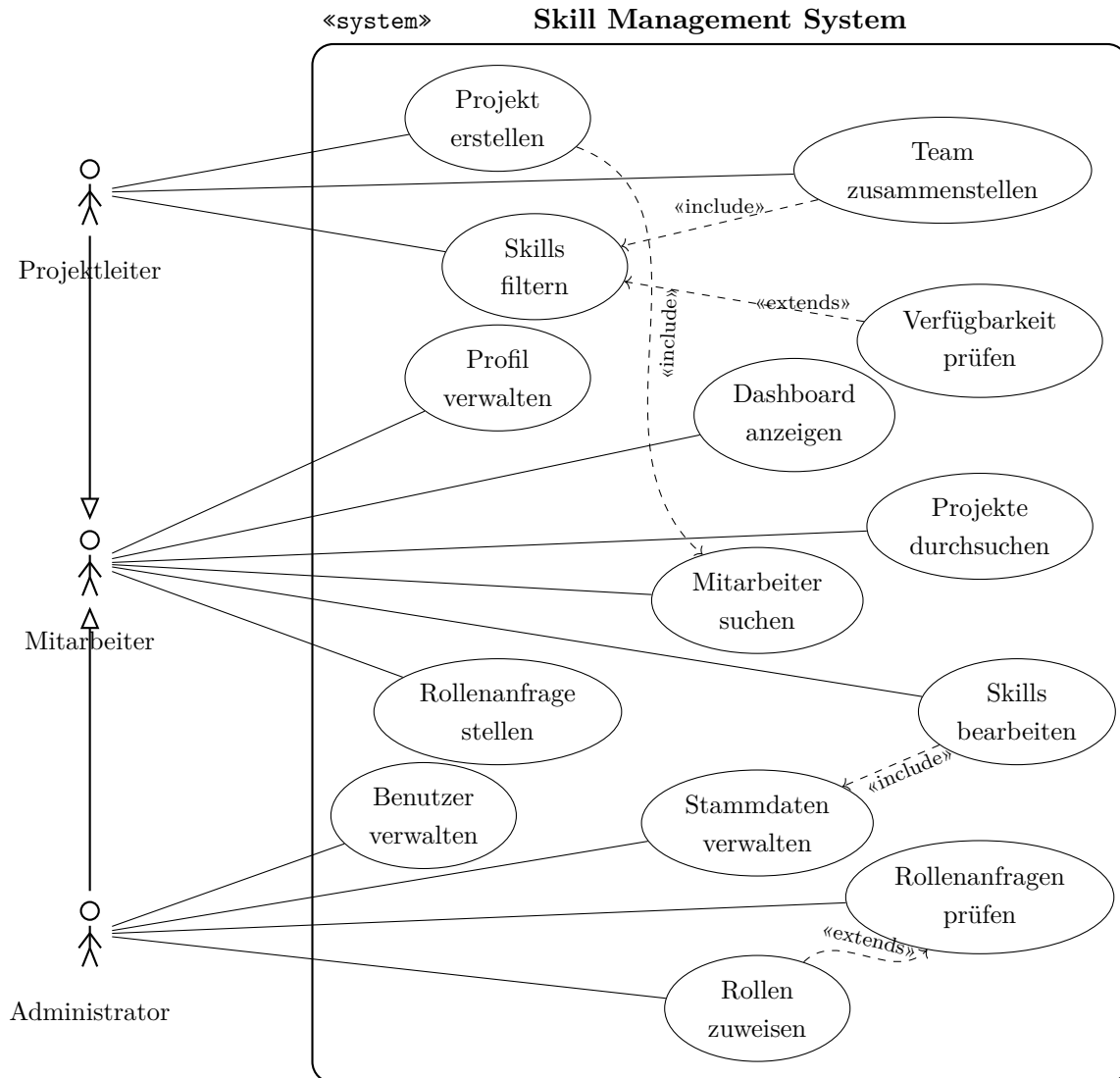


Abbildung 1.1: Use-Case-Diagramm des Skill Management Systems

Das System nutzt Keycloak[6] als zentralen Identity Provider (IdP), wodurch die Implementierung vereinfacht wird und eine spätere Integration in das unternehmensweite Single Sign-On (SSO) der EDAG ermöglicht wird. Alle im Use-Case-Diagramm dargestellten Aktionen setzen zudem eine erfolgreiche Authentifizierung und rollenbasierte Autorisierung über Keycloak[6] voraus.

1.4 Projektnutzen

Die Automatisierung reduziert den Aufwand pro Projektbesetzung von 2,5 Stunden auf 15 Minuten (90% Reduktion), was jährlich 8.437,50 € einspart (siehe Abschnitt 2.4.3)[1, 16]. Zusätzliche Vorteile:

- Transparente Kompetenzübersicht für strategische Personalplanung
- Datenbasierte Personalauswahl verbessert Projekterfolgsrate
- Sichtbarkeit individueller Kompetenzen erhöht Mitarbeiterzufriedenheit
- Fundierte Grundlage für gezielte Weiterbildungsmaßnahmen

1.5 Projektabgrenzung

Das MVP ist als internes Tool für die Abteilung Mobility IT konzipiert. Die Architektur unterstützt eine spätere unternehmensweite Ausweitung.

Im Projektumfang: Webbasierte Anwendung mit Skill-/Projektverwaltung, rollen-basiertem Zugriff, Keycloak-Integration und REST-API.

Ausgeschlossen: Native mobile Apps, HR-System-Integrationen, erweiterte Analytics und Reporting-Funktionen.

Kapitel 2

Projektplanung

2.1 Projektphasenmodell

Das Projekt wurde nach einem sequenziellen Phasenmodell mit vier Hauptphasen strukturiert. Die Gesamtprojektdauer umfasste 80 Stunden und erstreckte sich über den Zeitraum vom 1. Oktober 2025 bis zum 1. Dezember 2025. Jede Phase baute auf den Ergebnissen der vorherigen auf und war durch klar definierte Meilensteine abgegrenzt.

2.1.1 Phasenübersicht

Die folgende Tabelle 2.1 stellt die vier Projektphasen sowie deren Zeitanteile dar und dient als Grundlage für die detaillierte Ablauf- und Ressourcenplanung.

Phase	Dauer	Zentrale Aktivitäten
Analysephase	8h	Erhebung des Projektumfelds, IST-Analyse, Strukturierung der Anforderungen, Prüfung datenschutzrelevanter Rahmenbedingungen
Planungsphase	12h	Entwurf der Systemarchitektur, Technologieauswahl, Definition des Datenmodells, Erstellung erster UI-Mockups, Wirtschaftlichkeitsbetrachtung
Durchführungsphase	44h	Aufsetzen der Entwicklungsumgebung, Implementierung der Datenbank- und Backendschicht, Umsetzung des Frontends, Systemintegration, Einrichtung der CI/CD-Pipeline
Abschlussphase	16h	Planung und Durchführung der Systemtests, Qualitätssicherung, Erstellung der technischen Dokumentation und Projektdokumentation
Gesamt	80h	

Tabelle 2.1: Phasenmodell mit Zeitverteilung

Die detaillierte Aufgliederung aller 21 Arbeitspakete mit zeitlichem Ablauf und Abhängigkeiten ist in Tabelle A.1 im Anhang dokumentiert.

2.2 MVP-Definition und Priorisierung

Das Projekt folgt dem MVP-Ansatz, um innerhalb des begrenzten Zeitrahmens von 80 Stunden ein funktionsfähiges System zu liefern, das die kritischsten Anforderungen abdeckt.

2.2.1 MVP-Kernfunktionen

Die MVP-Definition fokussiert auf die wesentlichen Funktionen, die den größten Mehrwert für die Personalauswahl bieten:

Feature	Priorität	Begründung
Mitarbeitersuche mit Skill-Filtern	Must-Have	Kernfunktion für Projektbesetzung
Profilverwaltung mit Skills	Must-Have	Datenbasis für Suche
Projektverwaltung (Manager)	Must-Have	Zentrale Dokumentation
Authentifizierung (OAuth2)	Must-Have	Sicherheitsanforderung
Rollenbasierte Zugriffskontrolle	Must-Have	Berechtigungskonzept
Dashboard mit Statistiken	Should-Have	Verbesserte UX
Activity-Logging	Should-Have	Audit-Trail
Stammdatenverwaltung (Admin)	Should-Have	Systemkonfiguration

Tabelle 2.2: MVP-Feature-Priorisierung nach MoSCoW-Methode[23]

2.2.2 Abgrenzung: Nicht im MVP enthalten

Folgende Features wurden bewusst für spätere Versionen zurückgestellt:

- **Skill-Gap-Analyse:** Automatische Identifikation fehlender Kompetenzen
- **Weiterbildungsempfehlungen:** KI-gestützte Trainingsvorschläge
- **Benachrichtigungssystem:** Push-Notifications und E-Mail-Alerts
- **Erweiterte Analytics:** Skill-Heatmaps, Trend-Analysen
- **Export-Funktionen:** PDF-Reports, Excel-Export
- **Integration mit Drittsystemen:** Jira, Confluence, LDAP

Diese Priorisierung stellt sicher, dass das MVP innerhalb des Projektzeitraums vollständig funktionsfähig und testbar ist, während zukünftige Erweiterungen systematisch geplant werden können.

2.2.3 Gantt-Diagramm

Abbildung 2.1 visualisiert den zeitlichen Ablauf des Projekts über acht Projektwochen mit den definierten Meilensteinen.

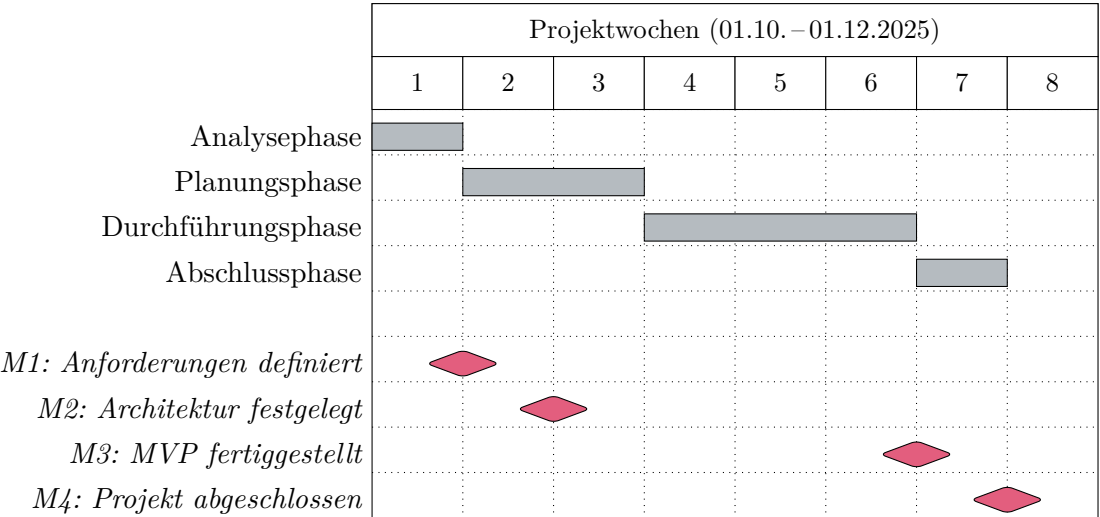


Abbildung 2.1: Gantt-Diagramm des Projektzeitplans

2.3 Ressourcenplanung

Das Projekt wurde vom Auszubildenden Marvin Eschenbach durchgeführt mit Unterstützung des betrieblichen Betreuers Christian Henning (Beratung, Code-Reviews, Abnahme). Die Entwicklung erfolgte auf einem Entwicklungsrechner mit Docker- und Kubernetes-Unterstützung[13, 7] sowie Zugriff auf die EDAG-Infrastruktur (Bitbucket[24]).

Sämtliche verwendeten Softwarewerkzeuge sind entweder Open Source oder unternehmensseitig lizenziert; zusätzliche Kosten entstehen dadurch nicht. Der vollständige Technologie-Stack ist in Tabelle A.2 im Anhang dokumentiert.

2.4 Wirtschaftlichkeitsbetrachtung

2.4.1 Kostenplanung

Die Projektumsetzung verursacht ausschließlich Personalkosten. Die notwendige Infrastruktur sowie alle Softwarewerkzeuge stehen bereits bereit. Die detaillierte Kostenaufstellung ist in Tabelle 2.3 dargestellt.

Position	Stundensatz	Aufwand	Kosten
Entwicklungskosten			
Auszubildender (Entwicklung)	35,00 €[25]	70h	2.450,00 €
Auszubildender (Dokumentation)	35,00 €[25]	10h	350,00 €
<i>Zwischensumme Entwicklung</i>			2.800,00 €
Betreuung und Reviews			
Projektbetreuer (Beratung)	75,00 €[26]	3h	225,00 €
Projektbetreuer (Code-Reviews)	75,00 €[26]	2h	150,00 €
<i>Zwischensumme Betreuung</i>			375,00 €
Infrastruktur (anteilig)			
Entwicklungsrechner	–	–	0,00 €
Softwarelizenzen	–	–	0,00 €
Kubernetes-Infrastruktur	–	–	0,00 €
<i>Zwischensumme Infrastruktur</i>			0,00 €
Gesamtinvestition			3.175,00 €

Tabelle 2.3: Detaillierte Kostenplanung

2.4.2 Make-or-Buy-Entscheidung

Kommerzielle Skill-Management-Systeme kosten 5–15 € pro Nutzer/Monat[27], was bei 200 Mitarbeitern jährlich 12.000–36.000 € entspricht. Die Eigenentwicklung verursacht einmalig 3.175 €.

Die **Make-Variante** wurde gewählt aufgrund: langfristig geringerer Kosten, vollständiger Datenkontrolle (DSGVO[28]), optimaler Integration in EDAG-Infrastruktur und maximaler Flexibilität ohne Vendor-Abhängigkeit. Der detaillierte Vergleich findet sich in Tabelle A.5.

2.4.3 Nutzenanalyse

Die Nutzenanalyse vergleicht den aktuellen manuellen Prozess (IST) mit dem automatisierten System (SOLL):

IST-Prozess: 2,5 Stunden pro Projektbesetzung = 125 Stunden/Jahr = 9.375 € Kosten[1, 16]

SOLL-Prozess: 15 Minuten pro Projektbesetzung = 12,5 Stunden/Jahr = 937,50 € Kosten

Einsparung: 112,5 Stunden (90%) = **8.437,50 € jährlich**

Die detaillierte Aufschlüsselung der Aktivitäten findet sich in den Tabellen A.3 und A.4 im Anhang.

2.4.4 Amortisationsrechnung

Die statische Amortisationsrechnung ermittelt den Zeitpunkt, zu dem die Investitionskosten durch eingesparte Kosten ausgeglichen sind:

$$\text{Amortisationsdauer} = \frac{\text{Investitionskosten}}{\text{jährliche Einsparung}} = \frac{3.175 \text{ EUR}}{8.437,50 \text{ EUR}} = 0,38 \text{ Jahre} \approx \mathbf{4,5 \text{ Monate}}$$

Der **Return on Investment (ROI)** beträgt 166% im ersten Jahr. Die Amortisation erfolgt bereits nach 4,5 Monaten.

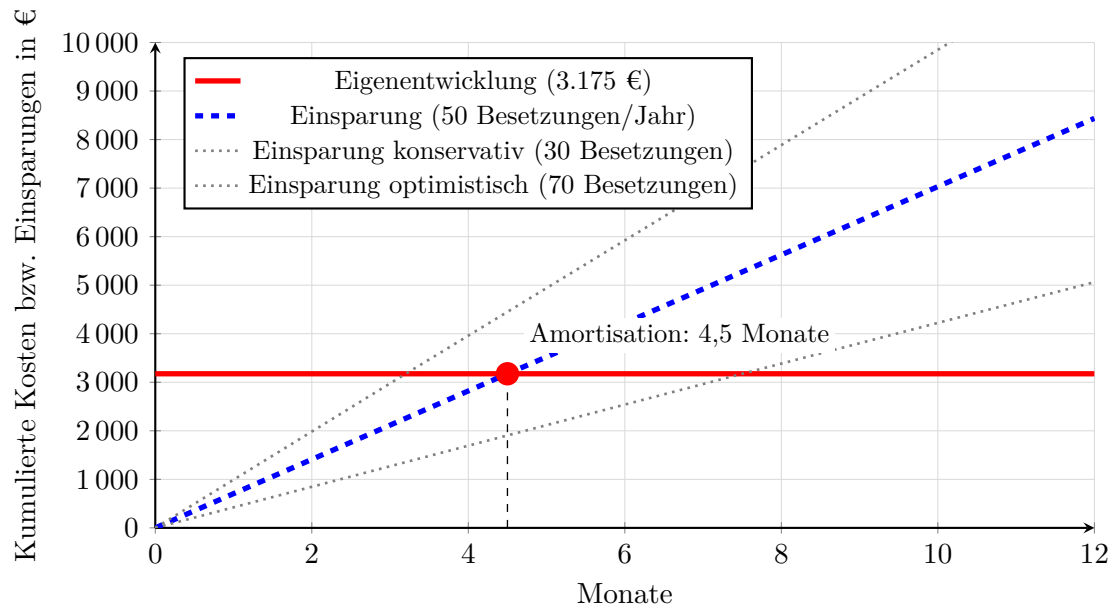


Abbildung 2.2: Amortisationsanalyse über 12 Monate

Selbst im konservativen Szenario (30 Besetzungen/Jahr) amortisiert sich die Investition nach 7,5 Monaten.

Kapitel 3

Projektdurchführung

3.1 Entwicklungsumgebung

Die Entwicklungsumgebung wurde containerisiert mit Rancher Desktop[29] für Kubernetes[7]-Support aufgesetzt. Als Integrated Development Environments (IDEs) kamen IntelliJ IDEA (Backend) und WebStorm (Frontend)[30, 31] zum Einsatz. Als KI-gestütztes Entwicklungs-Hilfsmittel wurde GitHub Copilot[32] eingesetzt, das sowohl beim Schreiben von Code als auch bei der Codebase-Analyse unterstützte und dadurch die Entwicklungsgeschwindigkeit signifikant erhöhte.

Lokale Services (Kubernetes Namespace `skill-management-system-dev`):

- PostgreSQL 15[5] (StatefulSet mit persistentem Volume)
- Keycloak 23[6] (Realm `skill-management`)
- Backend/Frontend lokal für Hot-Reload

Versionsverwaltung erfolgte über Bitbucket[24] mit Git-Flow-Strategie (`main`, `feat/*`-Branches). Das Mono-Repository enthält Backend (Spring Boot[4]), Frontend (Next.js[2]), Webhook-Extension (Keycloak[6]) und Kubernetes-Konfigurationen.

Die vollständige Repository-Struktur ist im Anhang, Listing A.1, dokumentiert. Die Commit-Historie folgt Conventional Commits-Konventionen[33] mit Commit Lint (siehe Abbildung A.8 im Anhang).

3.2 Datenbankdesign und -implementierung

3.2.1 Datenmodell

Das Datenmodell wurde in 3. Normalform nach relationalen Designprinzipien entworfen[34]. Die Datenbank umfasst 14 Tabellen (`users`, `employee_profiles`, `skills`, `skill_categories`, `projects` etc.) und nutzt *Flyway*-Migrationen für die Versionskontrolle. Das ER-Diagramm (Abbildung A.13) zeigt die Kerneinheiten des Systems mit ihren Primär- und Fremdschlüsseln für einen schnellen strukturellen Überblick, während das vollständige Datenbank-Schema (Abbildung A.14) im Anhang (Abschnitt A.10) alle Tabellen, Attribute und Relationen detailliert darstellt.

Kernbeziehungen:

- **users** 1:1 **employee_profiles** (Profildaten)
- **n:m employee_skills** (Skill-Zuordnungen mit Proficiency 0–100)
- **n:m project_members** (Team-Zusammensetzungen)

Eine automatisierte Tabelle **user_statistics** aggregiert Echtzeit-Metriken über Database-Trigger.

3.2.2 Datenbank-Migration mit Flyway

Für das Datenbankschema-Management wurde Flyway[35] eingesetzt. Alle Schemaänderungen sind als versionierte SQL-Migrationsskripte dokumentiert (V1–V15). Dies gewährleistet reproduzierbare Deployments und nachvollziehbare Schemaänderungen über den gesamten Entwicklungszyklus.

Die 15 Migrationen decken folgende Bereiche ab:

- **V1–V4**: Basis-Tabellen (users, skills, projects, analytics)
- **V5–V8**: Trigger für Timestamps, Statistiken und Audit-Logging
- **V9**: Initial-Daten (200+ Skills, 55+ Positionen, 30 Standorte)
- **V10–V13**: Refactoring (Auto-Profile-Init, Project-Members, Team-Size)
- **V14–V15**: Role-Requests und erweiterte Activity-Logs

Indizes wurden strategisch auf Suchspalten (email, username) und alle Foreign Keys gesetzt. PL/pgSQL-Trigger wurden implementiert, die bei Änderungen automatisch Statistiken und Audit-Logs aktualisieren:

- **user_statistics**: Echtzeit-Berechnung von total_skills, average_skill_score, active_projects (V6)
- **activities**: Immutable Audit-Log für 30+ Event-Typen (SKILL_ADDED, PROJECT_CREATED, ROLE_REQUEST_APPROVED etc.) (V7, V15)
- **skill_development**: Monatliche Snapshots für Entwicklungs-Tracking (V8)
- **projects.team_size**: Automatische Berechnung basierend auf project_members (V12)

Auszüge der SQL-Migrationsskripte (V1–V15) sind im Anhang dokumentiert (siehe Tabelle A.7).

3.3 Backend-Implementierung

3.3.1 Architektur

Das Backend folgt dem Prinzip der Hexagonalen Architektur (Ports & Adapters)[36], um eine klare Trennung zwischen Geschäftslogik und technischen Details zu erreichen. Die Implementierung basiert auf Spring Boot 3.5.7 mit Java 25[4].

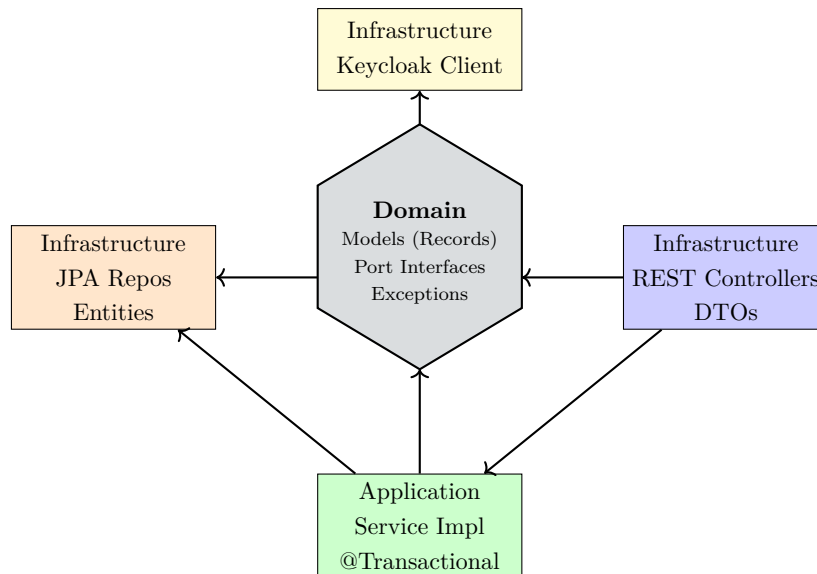


Abbildung 3.1: Hexagonale Architektur des Backend-Systems

3.3.2 Schichtenaufteilung

Domain-Schicht: Enthält die fachliche Kernlogik als immutable Java Records. Zentrale Klassen sind `User`, `Employee`, `ProfileSkill`, `Project` sowie Port-Interfaces wie `ProfileService` oder `EmployeeDiscoveryService`. Die im Anhang dargestellte Klassendiagramm-Übersicht (Abbildung A.6) visualisiert die wichtigsten Bestandteile der Domain, ohne jedoch sämtliche Details abzubilden.

Application-Schicht: Implementiert die Service-Interfaces und orchestriert die Geschäftslogik. So realisiert etwa `EmployeeDiscoveryServiceImpl` die Mitarbeitersuche unter Verwendung von `@Transactional`-Management und dynamischen JPA Specifications. Der Suchablauf ist im Aktivitätsdiagramm (Anhang, Abbildung A.5) dargestellt, während der vollständige Request-Flow über alle Architekturschichten hinweg im Sequenzdiagramm (Anhang, Abbildung A.2) visualisiert wird.

Infrastructure-Schicht: Implementiert die technischen Schnittstellen:

- **REST-Controller** (`infrastructure.web`): REST-Endpunkte mit Spring Web
- **JPA-Repositories** (`infrastructure.persistence`): Datenbankzugriff über Spring Data JPA mit Object-Relational Mapping (ORM)-basierten Entities sowie dynamischen Abfragen mittels JPA Specifications
- **Keycloak-Client** (`infrastructure.security`): User-Synchronisation via Keycloak Admin API

3.3.3 Zentrale API-Endpunkte

Die Representational State Transfer (REST)-Application Programming Interface (API)[37] folgt den RESTful-Prinzipien und ist vollständig mittels OpenAPI 3.0[38] dokumentiert.

Insgesamt umfasst das System über 30 Endpunkte, die in sieben funktionale Bereiche gegliedert sind:

- **Profilverwaltung** (/v1/profiles)
- **Mitarbersuche** (/v1/employees)
- **Projektverwaltung** (/v1/projects, /v1/management/projects)
- **Rollenanfragen** (/v1/role-requests)
- **Analytics** (/v1/analytics)
- **Stammdatenverwaltung** (/v1/options)
- **Administration** (/v1/admin)

Eine vollständige Übersicht sämtlicher Endpunkte inklusive HTTP-Methoden, Beschreibungen und Rollenberechtigungen befindet sich im Anhang (Abschnitt A.6.4, Tabelle A.10). Die API verwendet JSON Web Token (JWT)-basierte Authentifizierung[39] für alle geschützten Ressourcen.

Der vollständige Request-Flow durch alle Architekturschichten ist am Beispiel der Employee-Discovery-API in den Anhängen A.2 bis A.5 dargestellt. Die Implementierung demonstriert zentrale Clean-Architecture-Prinzipien wie Dependency Inversion, Data Transfer Object (DTO)-Mapping, dynamisches Query-Building mittels JPA Criteria API sowie N+1-Vermeidung durch Batch-Loading.

3.3.4 Authentifizierung und Autorisierung

Die Integration mit Keycloak[6] erfolgt über Spring Security OAuth2 Resource Server[40]. Spring Security validiert Access Tokens von Keycloak und erzwingt Rollen (USER/MANAGER/ADMIN) via `@PreAuthorize`-Annotations. Der detaillierte OAuth2 Token Introspection Flow ist in Abbildung A.1 im Anhang dargestellt. Ein Code-Beispiel für die rollenbasierte Autorisierung im Projektmanagement-Controller ist im Anhang, Listing A.10 dokumentiert.

Die `AuthorizationUtils`-Klasse unterstützt den "meAlias zur Referenzierung des aktuellen Users und prüft Zugriffsrechte auf Ressourcen-Ebene (z. B. User darf nur eigenes Profil ändern, Manager nur eigene Projekte modifizieren).

3.4 Frontend-Implementierung

3.4.1 Technologie-Stack und State Management

Das Frontend basiert auf Next.js 16 mit App Router (Server Components + CSR), TypeScript für Typsicherheit und Tailwind CSS für responsives Styling[2, 41, 42]. Die Benutzeroberfläche wurde unter Berücksichtigung des EDAG Corporate Designs gestaltet, um eine konsistente visuelle Integration in die Unternehmens-Systemlandschaft zu gewährleisten. Zentrale Bibliotheken: React 19[3], NextAuth.js[43] (OAuth2/OIDC mit Keycloak),

Redux Toolkit + RTK Query[44] (State Management mit Caching), shadcn/ui[45] (Radix UI-basierte Komponenten) und Recharts[46] für Skill-Visualisierung und Charts.

3.4.2 Komponentenstruktur

Die UI folgt einer feature-basierten Architektur[47], umgesetzt mit dem Next.js App Router für dateibasiertes Routing. Die Struktur trennt klar zwischen Routing (`app/`), wiederverwendbaren UI-Komponenten (`components/`), Feature-spezifischen Bildschirmen (`screens/`) sowie dem globalen State Management (`store/`). Dadurch werden Verantwortlichkeiten sauber gekapselt und Wiederverwendbarkeit wie auch Testbarkeit erhöht.

Das in Woche 2 erstellte UI-Mockup der Dashboard-Startseite (Anhang, Abbildung A.32) diente als visuelle Leitlinie während der Implementierung. Die final umgesetzten Screens umfassen Authentifizierung, Dashboard, Profilverwaltung, Mitarbeiter- und Projektsuche sowie Administrationsbereiche (Screenshots siehe Anhang, Abbildungen A.15 bis A.31).

Ein Beispiel für die Umsetzung eines Feature-Screens unter Verwendung von Custom Hooks und RTK Query ist im Anhang in Listings A.6 und A.7 dargestellt.

3.4.3 Kernfunktionen

Dashboard: Das Dashboard zeigt eine personalisierte Begrüßung, wichtige Kennzahlen zu Skills und Projekten sowie die zeitliche Entwicklung der eigenen Skills in einem Area-Chart. Zusätzlich werden die neuesten Aktivitäten und zuletzt bearbeiteten Projekte dargestellt. Ein Bereich mit Schnellaktionen ermöglicht den direkten Zugriff auf häufig genutzte Funktionen. Die Daten werden serverseitig initial geladen und anschließend clientseitig aktualisiert.

Skill-Verwaltung: Mitarbeitende können ihre Skills in einer strukturierten Ansicht nach Kategorien verwalten. Über ein modales Formular lassen sich neue Skills hinzufügen oder bestehende bearbeiten. Dabei können Kategorie, Skill, Erfahrungsjahre, letzter Einsatz sowie ein Proficiency-Score angepasst werden. Vorschlagslisten unterstützen die Auswahl, und clientseitige Prüfungen stellen konsistente Eingaben sicher. Der CRUD-Workflow ist im Sequenzdiagramm (Anhang, Abbildung A.3) dargestellt.

Mitarbeitersuche: Die Mitarbeitersuche kombiniert Freitextsuche mit mehreren Filterdimensionen: Verfügbarkeit, Skill-Kategorien, einzelne Skills, Standort sowie Mindest-Erfahrung in Jahren. Die Filterlogik basiert auf dynamisch generierten Filter-Optionen aus dem Backend. Ergebnisse werden in einem responsiven Grid dargestellt und können über Pagination effizient durchgeblättert werden. Die gesamte Filter- und Suchlogik ist mit lokalem UI-State gekoppelt und wird über einen dedizierten Hook (`useEmployeeDiscover`) gemanagt.

Projektverwaltung (Manager): Die Projektverwaltung ist eng mit der Mitarbeitersuche verknüpft. Manager können Projekte anlegen und anschließend Mitarbeiter in einem speziellen „Selection Mode“ auswählen.

Projekterstellung mit Mitarbeiterauswahl: Der Projekt-Erstellungs-Flow nutzt `sessionStorage` als Zwischenspeicher für Dialog-State während der Navigation zur Mitarbeitersuche. Die `Employee-Discovery`-Ansicht wird im `Selection Mode` um auswählbare Karten und einen `Selection-Banner` erweitert; nach Rückkehr werden Formulardaten und ausgewählte Mitarbeiter wiederhergestellt. Abbildung A.4 im Anhang zeigt den vollständigen Ablauf.

3.4.4 State Management

Das State Management erfolgt mit `Redux Toolkit`. `RTK Query` verwaltet Server-Daten mit automatischem Caching und `Background-Refetching`. API-Endpoints (`searchEmployees`, `getFilterOptions` etc.) werden deklarativ mit `createApi` und `baseQuery` definiert. Durch `providesTags` und `invalidatesTags` wird eine gezielte Cache-Invalidierung ermöglicht. Die vollständige Implementierung ist im Anhang in Listing A.8 dokumentiert.

3.5 Systemintegration

3.5.1 Keycloak-Anbindung und User-Synchronisation

Keycloak 23[6] dient als zentraler IdP mit dediziertem Realm `skill-management`. Die Konfiguration umfasst einen `Confidential Client` (`OAuth2 Authorization Code Flow`), Realm-Rollen (`user`, `admin`, `manager`) und eine `Access Token Lifespan` von 5 Minuten.

Die User-Synchronisation erfolgt ereignisbasiert über einen `Custom Event Listener` (`WebhookEventListenerProvider`). Dieser reagiert auf `REGISTER`-Events und `Admin-Events` (`CREATE/UPDATE/DELETE`) und sendet `HTTP-POST-Requests` mit `Basic Auth` an `KeycloakWebhookController`. Der `UserSyncServiceImpl` persistiert User-Daten transaktional in die `users`-Tabelle (vollständige Controller-Implementierung siehe Anhang, Listing A.9).

3.5.2 Rollenanfrage-Workflow (Self-Service)

Das System implementiert einen `Self-Service-Workflow` für Rollenanfragen, der es Benutzern ermöglicht, eigenständig erweiterte Berechtigungen (`MANAGER`-Rolle) anzufordern. Der Workflow minimiert administrativen Aufwand und schafft Transparenz durch automatisierte `Activity-Logs`.

Der Prozess umfasst drei Zustände (siehe Zustandsdiagramm im Anhang, Abbildung A.7):

- **PENDING:** User erstellt Rollenanfrage über UI (`/v1/role-requests`)
- **APPROVED:** Admin genehmigt Anfrage → Backend weist Rolle in Keycloak zu → `Activity-Log-Eintrag`
- **REJECTED:** Admin lehnt ab → `Status-Update` in DB → `Activity-Log-Eintrag`

3.6 Kubernetes-Deployment

Das System wird in einem lokalen Kubernetes-Cluster mit drei isolierten Namespaces betrieben: **skill-management-system-dev** (Keycloak, PostgreSQL für Entwicklung), **skill-management-system-prod** (Frontend, Backend, Keycloak, PostgreSQL), **skill-management-devops-prod** (Jenkins, SonarQube, Docker Registry, PostgreSQL). Traefik Ingress Controller exponiert Services über Hostnames (siehe Tabelle A.8 im Anhang für vollständige Übersicht). Deployment erfolgt über PowerShell-Skripte (**generate-secrets.ps1**, **deploy.ps1**) mit Kustomize Base/Overlay-Pattern für umgebungsspezifische Anpassungen.

Die Kubernetes-Architektur ist in Abbildung 3.2 visualisiert und zeigt die Namespace-Struktur, Service-Komponenten und Ingress-Routing.

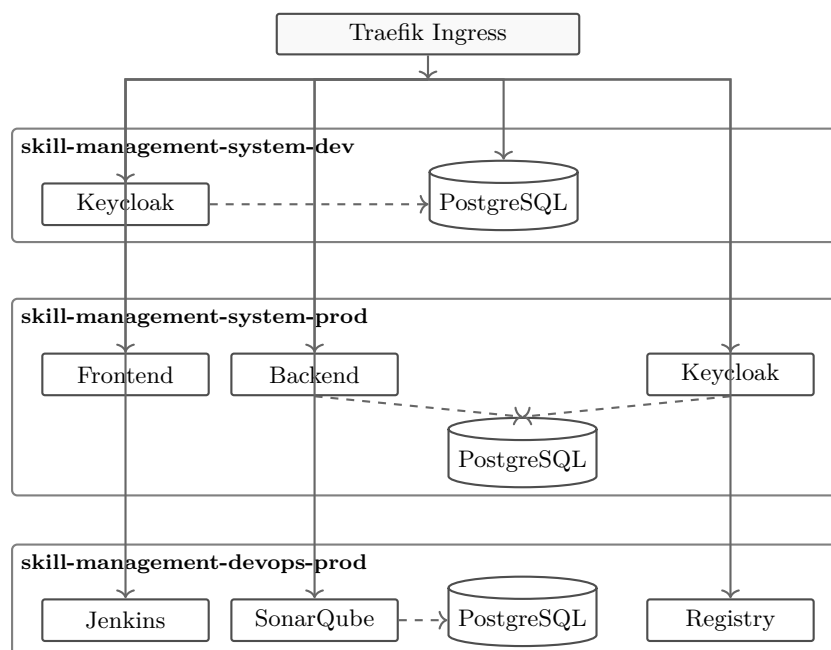


Abbildung 3.2: Kubernetes-Architektur: 3 Namespaces mit Services und Ingress-Routing

3.7 CI/CD-Pipeline

Die automatisierte CI/CD-Pipeline wurde als Jenkins Declarative Pipeline mit Kubernetes-Agents implementiert[8]. Sequenzielle Stages: **Checkout** (Bitbucket), **Backend Build & Analyze** (Maven `clean test package`, SonarQube[9]-Analyse, Quality Gate A-Rating), **Frontend Build & Analyze** (`npm ci && npm run build`, ESLint-Analyse, Quality Gate). Bei Branch `main`: **Build & Push Images** (parallele Docker[13]-Builds für Backend/Frontend, Push an Registry mit Tags `${BUILD_NUMBER}` und `latest`), **Trigger Deploy** (`kubectl apply` via `skill-management-deploy-Job`). Pipeline bricht bei Quality-Gate-Verletzung ab. Die vollständige Pipeline-Visualisierung ist im Anhang, Abbildung A.11 dargestellt. Die Jenkinsfile-Konfiguration ist in Listing A.11 dokumentiert.

3.8 Qualitätssicherung

Alle 15 Use Cases (siehe Anhang, Tabelle A.9) wurden in der Produktionsumgebung systematisch getestet. Die Tests umfassten alle Kernbereiche: Authentifizierung (Login/Logout, Rollenprüfung), Profilverwaltung (Skill-CRUD, Proficiency-Updates), Mitarbeiter-suche (Freitext-/Skill-/Standort-Filter), Projektverwaltung (Projekt-CRUD, Team-Zusammenstellung) sowie Analytics (Activity-Log). Edge-Cases wie fehlende Eingaben, ungültige Token und Netzwerk-Timeouts wurden validiert.

Drei Testpersonen aus der Zielgruppe (Projektleiter, Entwickler, Administrator) validierten die Benutzerfreundlichkeit. Feedback führte zu UI-Verbesserungen (Tooltips für Skill-Level, klarere Filterbeschriftungen).

Alle Feature-Branches durchliefen Code-Reviews durch den betrieblichen Betreuer (Clean Code[48], Architekturkonformität, Dokumentation). Pull Requests wurden nur nach Approval und erfolgreichem Quality Gate gemerged.

Kapitel 4

Projektergebnisse

4.1 Funktionale Systemvalidierung

Das entwickelte MVP deckt alle definierten Kernfunktionen ab und wurde erfolgreich getestet. Der MVP-Ansatz ermöglichte die Fokussierung auf die wesentlichen Features für die Projektbesetzung. Ergänzend zu dieser Projektdokumentation wurden umfassende Benutzer- und Entwicklerdokumentationen erstellt (siehe Anhang, Abbildung A.33), die Auszüge zur funktionalen Übersicht aus der Benutzerdokumentation (Abbildung A.34) sowie zur Architektur-Bausteinsicht aus der Entwicklerdokumentation (Abbildung A.35) enthalten.

4.1.1 Testabdeckung

Alle 15 Use Cases (siehe Anhang, Tabelle A.9) wurden systematisch getestet und validiert. Die Systemtests wurden manuell mit Fokus auf funktionale Korrektheit, Benutzerführung und Security durchgeführt. Alle kritischen Pfade wurden erfolgreich validiert, einschließlich der Kernbereiche Authentifizierung & Profile, Discovery & Suche, Projektverwaltung sowie Analytics. Edge-Cases und Fehlerszenarien wurden ebenfalls geprüft.

4.1.2 Code-Qualitätsmetriken

SonarQube[9]-Analysen ergaben nach 15 Builds durchgehend A-Ratings (Maintainability-/Security/Reliability), 7.800+ LOC Backend, 10.500+ LOC Frontend, < 2% Duplizierungen (Details siehe Anhang, Tabelle A.6 und Abbildungen A.9, A.10). Alle Quality Gates wurden erfüllt; Pipeline-Konfiguration mit `abortPipeline: true` stellte sicher, dass nur qualitativ hochwertiger Code deployed wurde.

4.1.3 Performance-Ergebnisse

API-Antwortzeiten: < 200 ms für 95% aller Requests (Browser DevTools, Anforderung aus Kapitel 1.3.2 erfüllt). **Frontend-Ladezeit:** < 2s initial, < 300 ms Navigation durch Next.js[2] App Router. Lighthouse[49]-Audit bestätigt Performance-Score 100/100 (First Contentful Paint 0.3s, Largest Contentful Paint 0.4s, siehe Anhang Abbildung A.12). Da-

tenbankabfragen optimiert durch Indizes und JPA Specifications[50]. Docker-Images: Backend 459 MB, Frontend 1.58 GB (Multi-Stage Builds[51]).

4.2 Soll-Ist-Vergleich

4.2.1 Zeitliche Zielerreichung

Phase	Geplant	Tatsächlich	Abweichung
Analysephase	8h	7h	-1h
Planungsphase	12h	13h	+1h
Durchführungsphase	44h	45h	+1h
Abschlussphase	16h	15h	-1h
Gesamt	80h	80h	0h

Tabelle 4.1: Soll-Ist-Vergleich der Projektphasen

Das Projekt wurde exakt im geplanten Zeitrahmen abgeschlossen. Geringfügige Abweichungen in einzelnen Phasen kompensierten sich gegenseitig:

- **Analysephase (-1h):** IST-Analyse effizienter durch strukturierte Interviews
- **Planungsphase (+1h):** Zusätzlicher Aufwand für Mockup-Erstellung und Wirtschaftlichkeitsberechnung
- **Durchführungsphase (+1h):** Keycloak-Webhook-Integration benötigte mehr Zeit als geplant
- **Abschlussphase (-1h):** Dokumentation parallel zur Entwicklung reduzierte Abschlussaufwand

4.2.2 Funktionale Zielerreichung

Anforderung (vgl. Kapitel 3)	Status
Personalisiertes Dashboard mit Skill-Entwicklung	✓ Vollständig
Profilverwaltung mit Selbsteinschätzung (0–100)	✓ Vollständig
Mitarbeiter-/Skill-/Projektsuche	✓ Vollständig
Projektmanagement (Manager): Anlegen, Teams zusammenstellen	✓ Vollständig
Benutzerverwaltung (Admin): Rollenzuweisung	✓ Vollständig
Stammdatenverwaltung (Admin): Skills, Kategorien, Standorte	✓ Vollständig
Authentifizierung über OAuth2/OIDC mit Keycloak[6]	✓ Vollständig
Rollenbasierte Zugriffskontrolle (USER/MANAGER/ADMIN)	✓ Vollständig
Antwortzeiten < 2s für Suchanfragen[18]	✓ Vollständig
Responsive UI konform zu WCAG 2.1 AA[17]	✓ Vollständig
Kubernetes-Deployment mit CI/CD-Pipeline[7, 8]	✓ Vollständig

Tabelle 4.2: Funktionale Anforderungen – Zielerreichung

Alle definierten Anforderungen aus Kapitel 3 wurden vollständig umgesetzt. Das MVP befindet sich im finalen Teststadium und erfüllt alle priorisierten Must-Have- und Should-Have-Kriterien (siehe Tabelle 2.2).

4.2.3 Wirtschaftliche Zielerreichung

Die wirtschaftlichen Ziele wurden erreicht. Die Amortisationsrechnung aus Kapitel 2.4.4 zeigt folgende Ergebnisse bei 50 Projektbesetzungen pro Jahr:

- **IST-Prozess** (siehe Tabelle A.3): $125 \text{ Stunden} \times 75 \text{ EUR/h} = 9.375 \text{ EUR}$
- **SOLL-Prozess** (siehe Tabelle A.4): $12,5 \text{ Stunden} \times 75 \text{ EUR/h} = 937,50 \text{ EUR}$
- **Jährliche Einsparung: 8.437,50 EUR**

Die Entwicklungskosten von 3.175 EUR (siehe Tabelle 2.3) amortisieren sich nach Berechnung in Kapitel 2.4.4 innerhalb von **4,5 Monaten**. Der ROI beträgt 166% im ersten Jahr.

4.3 Systemdemonstration

Das Kernszenario demonstriert den vollständigen Workflow: Manager authentifiziert sich via Keycloak OAuth2/OIDC, nutzt Multi-Filter-Suche (Skills, Standort, Verfügbarkeit) zur Mitarbeiterauswahl, erstellt Projekt im Selection Mode mit Positionszuweisung, API-Request erstellt automatisch Projekt inkl. Team-Zuordnungen und Activity-Log. Der Workflow reduziert die Projektbesetzungszeit von 2,5 Stunden auf **15 Minuten** (90% Reduktion) durch zentralisierte Skill-Datenbank und automatisierte Teamzusammenstellung. Das Dashboard bietet personalisierte Insights (Skill-Entwicklung als Area-Chart, Projekt-Statistiken, Activity Feed, Schnellaktionen) zur Identifikation von Skill-Gaps und Ableitung von Weiterbildungsbedarfen.

4.4 Abweichungen und Anpassungen

Technische Anpassungen: Keycloak-Webhook statt zeitgesteuerter Batch-Sync (Echtzeit-Synchronisation), Redux Toolkit + RTK Query statt TanStack Query (vereint Server State Caching und komplexes Client State Management[44]), Kustomize statt Helm (leichtgewichtigeres Overlay-Pattern dev/prod).

Funktionale Erweiterungen: Rollenansfrage-Workflow (Self-Service statt Admin-Zuweisung), Skill Development Tracking (über monatliche Snapshots in `skill_development`-Tabelle), Project Members Refactoring (dedizierte `project_members`-Tabelle mit Position-Zuordnung). Alle Anpassungen verbesserten die Systemqualität ohne den Projektzeitrahmen zu überschreiten.

Kapitel 5

Fazit

5.1 Projektzusammenfassung

Das Skill Management System wurde erfolgreich als MVP und moderne Cloud-Native-Lösung für die Abteilung *Mobility IT* der EDAG Engineering GmbH entwickelt. Das als interne Lösung konzipierte MVP adressiert die identifizierten Schwachstellen des manuellen Projektbesetzungsprozesses durch eine zentrale, durchsuchbare Skill-Datenbank mit 200+ technischen Kompetenzen und umfassender Projektverwaltung.

5.1.1 Zentrale Ergebnisse

Die Projektdurchführung verlief planmäßig innerhalb des 80-Stunden-Budgets (1. Oktober – 1. Dezember 2025). Alle definierten funktionalen und nicht-funktionalen Anforderungen wurden vollständig umgesetzt:

- **Technische Architektur:** Moderne Three-Tier-Architektur mit Next.js 16[2] Frontend, Spring Boot 3.5[4] Backend nach Hexagonaler Architektur, PostgreSQL 15[5] mit 15 Flyway-Migrationen[35]
- **Security & DevOps:** OAuth2/OIDC-Integration mit Keycloak 23[6] (OAuth2-Authentifizierung und Autorisierung), Kubernetes-Deployment[7], CI/CD-Pipeline mit Jenkins[8] und SonarQube[9] Quality Gates (Maintainability/Security Rating A)
- **Code-Qualität:** 18.300+ LOC (Backend 7.800+, Frontend 10.500+), Intuitive UI mit WCAG 2.1 AA[17], responsive Design, Internationalisierung (DE/EN)

5.2 Zielerreichung

5.2.1 Wirtschaftliche Ziele

Die wirtschaftlichen Projektziele wurden vollständig erreicht. Das System amortisiert sich nach 4,5 Monaten bei einem ROI von 166% nach 12 Monaten (detaillierte Berechnung siehe Kapitel 2.4 und 2.4.4). Die Zeitreduktion von 2,5 Stunden auf 15 Minuten pro Projektbesetzung entspricht einer Effizienzsteigerung von 90%. Neben den quantifizierten Einspa-

rungen bietet das System strategische Mehrwerte wie verbesserte Skill-Transparenz und fundierte Weiterbildungsplanung.

5.2.2 Qualitätsziele

Alle Qualitätsziele wurden erfüllt oder übertroffen (siehe Anhang, Tabelle A.6):

- **Maintainability:** SonarQube[9] Rating A
- **Security:** Rating A, keine Security Hotspots, sichere OAuth2-Integration[19]
- **Performance:** API-Antwortzeiten < 200 ms (95. Perzentil), Frontend-Ladezeit < 2s[18]
- **Usability:** Intuitive Navigation, kein Schulungsaufwand, barrierefreie Bedienung nach WCAG 2.1 AA[17]

5.3 Reflexion und Lernerfahrungen

Technische Herausforderungen: Die Implementierung der Hexagonalen Architektur im Backend erforderte initiales Refactoring, zahlte sich jedoch durch hohe Testbarkeit aus. Der Next.js 16 App Router[2] stellte Herausforderungen bei der Balance zwischen Server Components und Client Components; die ereignisbasierte Keycloak-Webhook-Integration erforderte tiefes SPI-Verständnis; Kustomize Base/Overlay-Pattern für drei Namespaces (dev/prod/devops) erforderte Einarbeitung. Die parallele Dokumentation während der Entwicklung reduzierte Abschlussaufwand; der MVP-Ansatz mit Feature-Priorisierung verhinderte Scope Creep.

Fachliche Kompetenzerweiterung: Das Projekt ermöglichte praxisnahe Anwendung moderner Praktiken: Clean Architecture (Hexagonale Architektur), Cloud-Native Development (Kubernetes, Container-Orchestrierung), DevOps (CI/CD-Pipeline, SonarQube, Docker Multi-Stage Builds), Security (OAuth2/OIDC, JWT-Validierung), Fullstack-Entwicklung (Spring Boot, React, PostgreSQL, API-Design).

5.4 Ausblick und Weiterentwicklung

Das entwickelte System bietet eine solide Basis für zukünftige Erweiterungen. **Funktional:** Skill-Gap-Analyse, KI-gestützte Weiterbildungsempfehlungen, Benachrichtigungssystem, erweiterte Analytics (Skill-Heatmaps, Team-Kompetenz-Matrix), LDAP/AD-Integration, Jira/Confluence-Integration, Export-Funktionen (PDF-Reports, Excel). **Technisch:** Redis-Caching[52] für Filter-Optionen, Lazy Loading, Lighthouse[49]-Tests, Playwright[53] E2E-Tests, Prometheus[54]/Grafana[55]-Monitoring, Jaeger[56]-Tracing, PostgreSQL-Backups[57] mit Point-in-Time-Recovery, Multi-Region-Deployment.

5.5 Schlusswort

Das entwickelte Skill Management System demonstriert die erfolgreiche Umsetzung moderner Cloud-Native-Architekturen im Enterprise-Kontext. Der MVP-Ansatz ermöglichte die zeitgerechte Lieferung eines funktionsfähigen Systems mit den wichtigsten Features. Die Kombination aus hexagonalem Backend-Design[36], server-side-rendered Frontend und Kubernetes[7]-Orchestrierung resultiert in einem wartbaren, skalierbaren und sicheren System.

Die wirtschaftliche Amortisation nach 4,5 Monaten und der ROI von 166% nach 12 Monaten (siehe Kapitel 2.4.4) belegen den Mehrwert für die Organisation. Darüber hinaus schafft das System strategische Vorteile durch verbesserte Transparenz über verfügbare Kompetenzen, datengetriebene Teamzusammenstellung und fundierte Weiterbildungsplanung. Die während der Projektdurchführung erworbenen Kompetenzen in Clean Architecture, DevOps-Praktiken und Cloud-Native Development bilden eine solide Grundlage für zukünftige Enterprise-Projekte. Die konsequente Anwendung von Best Practices resultierte in qualitativ hochwertigem, wartbarem Code, der bereit für den Produktiveinsatz ist.

Anhang A

Anhang

A.1 Projektplanung & Wirtschaftlichkeit

A.1.1 Detaillierte Zeitplanung

Nr.	Arbeitspaket	Dauer	Anfang	Ende	Vorg.
Analysephase (8h)					
1	Projektumfeld analysieren	2h	01.10.25	01.10.25	–
2	IST-Analyse durchführen	2h	02.10.25	02.10.25	1
3	Anforderungen definieren	2h	03.10.25	03.10.25	2
4	Datenschutz klären	2h	04.10.25	04.10.25	3
Planungsphase (12h)					
5	Systemarchitektur entwerfen	3h	07.10.25	07.10.25	4
6	Technologien auswählen	2h	08.10.25	08.10.25	5
7	Datenmodell erstellen	2h	10.10.25	10.10.25	6
8	UI-Mockups erstellen	2h	11.10.25	11.10.25	7
9	Wirtschaftlichkeit berechnen	2h	14.10.25	14.10.25	8
10	Zeitplan finalisieren	1h	15.10.25	15.10.25	9
Durchführungsphase (44h)					
11	Entwicklungsumgebung einrichten	4h	20.10.25	20.10.25	10
12	Datenbank implementieren	4h	21.10.25	21.10.25	11
13	Backend-API entwickeln	10h	22.10.25	24.10.25	12
14	Frontend entwickeln	14h	27.10.25	31.10.25	13
15	Systemintegration durchführen	6h	03.11.25	04.11.25	14
16	CI/CD-Pipeline einrichten	6h	05.11.25	05.11.25	15
Abschlussphase (16h)					
17	Systemtests durchführen	3h	10.11.25	10.11.25	16
18	Code-Qualität prüfen	2h	11.11.25	11.11.25	17
19	Fehler korrigieren	3h	12.11.25	12.11.25	18
20	Technische Dokumentation erstellen	4h	13.11.25	13.11.25	19
21	Projektdokumentation erstellen	4h	14.11.25	14.11.25	20

Tabelle A.1: Detaillierte Zeitplanung des Projekts

A.1.2 Technologie-Stack

Tool	Einsatzbereich	Lizenz
Backend		
Java 25 / Spring Boot 3.5.7	Backend-Framework	Apache 2.0
Maven	Build-Management	Apache 2.0
Frontend		
Next.js 16 / React 19	Fullstack-Framework	MIT
TypeScript	Typsichere Sprache	Apache 2.0
Tailwind CSS	CSS-Framework	MIT
Redux Toolkit / RTK Query	State Management	MIT
shadcn/ui	UI-Komponenten	MIT
Recharts	Visualisierungen	MIT
Entwicklung & CI/CD		
IntelliJ IDEA / WebStorm	IDEs	Kommerziell
Git / Bitbucket	Versionsverwaltung	GPL/Kommerziell
Jenkins	CI/CD-Pipeline	MIT
SonarQube	Code-Qualität	AGPL/Enterprise
GitHub Copilot	KI-Entwicklungsassistent	Kommerziell
Postman	API-Testing	Enterprise
Infrastruktur		
Rancher Desktop	Kubernetes-Entwicklungsumgebung	Apache 2.0
Docker	Containerisierung	Apache 2.0
Kubernetes	Container-Orchestrierung	Apache 2.0
PostgreSQL 15	Datenbank	PostgreSQL Li-cense
Flyway	Datenbank-Migrationen	Apache 2.0
Keycloak 23	IAM	Apache 2.0
Traefik Ingress Controller	Load Balancer	MIT

Tabelle A.2: Übersicht der eingesetzten Technologien

A.1.3 Wirtschaftlichkeitsberechnung Details

A.1.3.1 IST-Prozess (manuell)

Aktivität	Zeit pro Vorgang	Jährlich
Excel-Tabellen durchsuchen	45 Min.	37,5h
E-Mail-Abstimmung mit Teamleitern	60 Min.	50,0h
Verfügbarkeit klären	30 Min.	25,0h
Dokumentation erstellen	15 Min.	12,5h
Gesamt	2,5h	125,0h

Tabelle A.3: IST-Prozess bei 50 Projektbesetzungen pro Jahr

A.1.3.2 SOLL-Prozess (automatisiert)

Aktivität	Zeit pro Vorgang	Jährlich
System-Suche mit Filtern	5 Min.	4,2h
Verfügbarkeit prüfen (automatisch)	3 Min.	2,5h
Kandidatenauswahl	5 Min.	4,2h
Dokumentation (automatisch)	2 Min.	1,7h
Gesamt	15 Min.	12,5h

Tabelle A.4: SOLL-Prozess mit Skill-Management-System

A.1.4 Make-or-Buy-Vergleich

Kriterium	Buy (Lizenzkauf)	Make (Eigenentwicklung)
Kosten	12.000–36.000 € jährlich (wiederkehrend)	3.175 € einmalig
Datensicherheit	Externe Cloud-Speicherung mit Abhängigkeit vom Anbieter	Vollständige interne Datenkontrolle und DSGVO-Konformität
Integration	Eingeschränkte Standard-Schnittstellen	Nahtlose Einbindung in bestehende EDAG-Systemlandschaft
Anpassbarkeit	Stark begrenzt auf Konfigurationsoptionen	Vollständig individuell an Unternehmensprozesse anpassbar
Abhängigkeit	Hohes Risiko eines Vendor Lock-in	Keine externen Abhängigkeiten, volle Kontrolle

Tabelle A.5: Make-or-Buy-Vergleich

A.2 Architektur & Implementierung

A.2.1 Repository-Struktur

Das Mono-Repository wurde mit folgender Struktur organisiert:

Listing A.1: Projektstruktur des Repositories

```

1 skill-management-system/
2 |-- backend/                                # Spring Boot Backend
3 |   |-- .idea/
4 |   |-- .mvn/
5 |   |-- logs/
6 |   |-- src/
7 |       |-- main/java/com/edag/skillmanagementsystem/
8 |           |-- application/
9 |           |-- config/
10 |           |-- security/
11 |           |-- service/
12 |           |-- domain/
13 |           |-- exception/
14 |           |-- model/
15 |           |-- port/

```

```

16 | | | '-- infrastructure/
17 | | | |-- adapter/
18 | | | |-- properties/
19 | | | '-- validation/
20 | | |-- resources/
21 | | '-- test/
22 | |-- Dockerfile
23 | |-- mvnw
24 | |-- mvnw.cmd
25 | '-- pom.xml
26 |
27 |-- frontend/                                # Next.js Frontend
28 | |-- .idea/
29 | |-- .next/
30 | |-- node_modules/
31 | |-- public/
32 | |-- src/
33 | | |-- app/
34 | | |-- components/
35 | | | |-- navigation/
36 | | | |-- providers/
37 | | | '-- ui/
38 | | |-- hooks/
39 | | |-- libs/
40 | | |-- locales/
41 | | |-- screens/
42 | | | |-- admin/
43 | | | |-- discover-employees/
44 | | | |-- discover-projects/
45 | | | |-- home/
46 | | | |-- my-activities/
47 | | | |-- my-projects/
48 | | | |-- profile/
49 | | | |-- project-details/
50 | | | '-- welcome/
51 | | |-- store/
52 | | |-- styles/
53 | | |-- types/
54 | | '-- utils/
55 | |-- Dockerfile
56 | |-- package.json
57 | |-- package-lock.json
58 | |-- tsconfig.json
59 | |-- next.config.ts
60 | |-- tailwind.config.ts
61 | '-- eslint.config.mjs
62 |
63 |-- webhook/                                # Keycloak Webhook Extension

```

```

64 | | -- .idea/
65 | | -- .mvn/
66 | | -- src/
67 | | | -- main/java/com/edag/keycloak/webhook/
68 | | | | -- WebhookEventListenerProvider.java
69 | | | | -- WebhookEventListenerProviderFactory.java
70 | | | -- resources/
71 | | -- target/
72 | | -- mvnw
73 | | -- mvnw.cmd
74 | | -- pom.xml
75 |
76 | -- k8s/                                # Kubernetes Deployments
77 | | -- app/
78 | | | -- base/
79 | | | | -- keycloak/
80 | | | | -- postgres/
81 | | | | -- kustomization.yaml
82 | | | -- overlays/
83 | | | | -- dev/
84 | | | | -- prod/
85 | | -- devOps/
86 | | | -- base/
87 | | | | -- docker-registry/
88 | | | | -- jenkins/
89 | | | | -- postgres-sonar/
90 | | | | -- sonarqube/
91 | | | | -- ingress.yaml
92 | | | | -- kustomization.yaml
93 | | | | -- namespace.yaml
94 | | | -- certs/
95 | | -- .env.dev
96 | | -- .env.prod
97 | | -- deploy.ps1
98 | | -- generate-secrets.ps1
99 |
100 | -- docs/
101 | | -- dev/
102 | | -- user/
103 | -- context.md
104 | -- Jenkinsfile
105 | -- .pre-commit-config.yaml
106 | -- .gitignore
107 | -- README.md
108 | -- setup.ps1

```

A.2.2 Code-Beispiele

A.2.2.1 Backend: Employee Discovery Request Flow

Der folgende vollständige Request-Flow demonstriert die Implementierung der Clean Architecture im Backend am Beispiel der Mitarbeitersuche (siehe Abbildung A.1 für den Sequenzablauf).

Controller Layer (Inbound Adapter)

Listing A.2: EmployeeDiscoveryController - REST Endpoint

```
1 @RestController
2 @RequestMapping("/v1/employees")
3 @RequiredArgsConstructor
4 @Tag(name = "Employee Discovery")
5 public class EmployeeDiscoveryController {
6
7     private final EmployeeDiscoveryService employeeDiscoveryService;
8     private final AuthorizationUtils authorizationUtils;
9
10    @GetMapping("/search")
11    public ResponseEntity<EmployeeSearchResponseDto> searchEmployees(
12        @ModelAttribute EmployeeSearchFiltersDto filtersDto,
13        @RequestParam(defaultValue = "0") int page,
14        @RequestParam(defaultValue = "20") int size,
15        @RequestParam(defaultValue = "name") String sortBy,
16        @RequestParam(defaultValue = "asc") String sortOrder) {
17
18        log.debug("REST request to search employees - filters: {},
19                page: {}, size: {}", filtersDto, page, size);
20
21        // DTO to Domain mapping
22        EmployeeSearchCriteria criteria = filtersDto.toDomain();
23
24        // Build pageable with sorting
25        Sort sort = sortOrder.equalsIgnoreCase("desc")
26            ? Sort.by(mapSortField(sortBy)).descending()
27            : Sort.by(mapSortField(sortBy)).ascending();
28        Pageable pageable = PageRequest.of(page, size, sort);
29
30        // Delegate to service
31        Page<Employee> results =
32            employeeDiscoveryService.searchEmployees(criteria, pageable
33                );
34
35        log.debug("Found {} of {} total employees",
36            results.getNumberOfElements(), results.getTotalElements());
```

```

37     return ResponseEntity.ok(EmployeeSearchResponseDto.from(results
38         ));
39 }
40 @GetMapping("/{userId}")
41 public ResponseEntity<EmployeeResponseDto> getEmployeeById(
42     @PathVariable("userId") String userId,
43     Principal principal) {
44
45     UUID resolvedId = authorizationUtils.resolveUserId(
46         userId, principal); // Supports "me" alias
47     Employee employee = employeeDiscoveryService.getEmployeeById(
48         resolvedId);
49     return ResponseEntity.ok(EmployeeResponseDto.from(employee));
50 }
51 }

```

Service Layer (Application)

Listing A.3: EmployeeDiscoveryServiceImpl - Business Logic

```

1  @Service
2  @RequiredArgsConstructor
3  @Transactional(readOnly = true)
4  public class EmployeeDiscoveryServiceImpl
5      implements EmployeeDiscoveryService {
6
7      private final EmployeeRepository employeeRepository;
8      private final OptionsRepository optionsRepository;
9
10     @Override
11     public Page<Employee> searchEmployees(
12         final EmployeeSearchCriteria criteria,
13         final Pageable pageable) {
14
15         log.debug("Searching employees with criteria: {} and pageable:
16             {}",
17                 criteria, pageable);
18
19         Page<Employee> results = employeeRepository.findByCriteria(
20             criteria, pageable);
21
22         log.info("Found {} employees out of {} total",
23             results.getNumberOfElements(), results.getTotalElements());
24
25         return results;
26     }

```

```

27  @Override
28  public Employee getEmployeeById(final UUID id) {
29      log.debug("Retrieving employee with id: {}", id);
30
31      Employee employee = employeeRepository.findById(id)
32          .orElseThrow(() -> new EmployeeNotFoundException(id));
33
34      log.info("Successfully retrieved employee: {}", id);
35      return employee;
36  }
37
38  @Override
39  public EmployeeFilterOptions getFilterOptions() {
40      log.debug("Retrieving employee filter options");
41
42      List<Location> locations =
43          optionsRepository.findAllActiveLocations();
44      List<Skill> skills = optionsRepository.findAllActiveSkills();
45      List<SkillCategory> categories =
46          optionsRepository.findAllActiveSkillCategories();
47
48      return new EmployeeFilterOptions(skills, locations, categories)
49          ;
50  }

```

Repository Layer (Outbound Adapter)

Listing A.4: EmployeeDataSource - Persistence Implementation

```

1  @Component
2  @RequiredArgsConstructor
3  public class EmployeeDataSource implements EmployeeRepository {
4
5      private final EmployeeJpaRepository employeeRepository;
6      private final EmployeeSkillJpaRepository employeeSkillRepository;
7
8      @Override
9      @Transactional(readOnly = true)
10     public Page<Employee> findByCriteria(
11         EmployeeSearchCriteria criteria, Pageable pageable) {
12
13         log.debug("Finding employees by criteria: {} with pageable: {}"
14             ,
15             criteria, pageable);
16
17         // Build JPA Specification from criteria
18         Specification<EmployeeEntity> spec =

```

```
18         EmployeeSpecification.fromCriteria(criteria);
19
20     Page<EmployeeEntity> employeePage =
21         employeeRepository.findAll(spec, pageable);
22
23     if (employeePage.isEmpty()) {
24         return employeePage.map(e ->
25             EmployeeMapper.entityToDomain(e, List.of()));
26     }
27
28     // Collect all user IDs from page
29     List<UUID> userIds = employeePage.stream()
30         .map(EmployeeEntity::getUserId)
31         .toList();
32
33     // Load top skills in one batch query (N+1 prevention)
34     Map<UUID, List<EmployeeSkillEntity>> topSkillsMap =
35         employeeSkillRepository.findTop3SkillsForEmployees(userIds)
36             .stream()
37             .collect(Collectors.groupingBy(
38                 skill -> skill.getEmployee().getUserId()));
39
40     // Map entities to domain objects
41     return employeePage.map(employee -> {
42         List<EmployeeSkillEntity> topSkills =
43             topSkillsMap.getOrDefault(employee.getUserId(), List.of()
44             );
45         return EmployeeMapper.entityToDomain(employee, topSkills);
46     });
47
48     @Override
49     @Transactional(readOnly = true)
50     public Optional<Employee> findById(UUID userId) {
51         log.debug("Finding employee by id: {}", userId);
52
53         return employeeRepository.findById(userId)
54             .map(employee -> {
55                 List<EmployeeSkillEntity> topSkills =
56                     employeeSkillRepository
57                         .findByEmployeeUserIdOrderByProficiencyScoreDesc(
58                             userId);
59                 return EmployeeMapper.entityToDomain(employee, topSkills)
60                     ;
61             });
62     }
```

Specification Pattern (Dynamic Query Building)

Listing A.5: EmployeeSpecification - JPA Criteria API

```

1  @NoArgsConstructor(access = AccessLevel.PRIVATE)
2  public final class EmployeeSpecification {
3
4      public static Specification<EmployeeEntity> fromCriteria(
5          final EmployeeSearchCriteria criteria) {
6
7          return (root, query, cb) -> {
8              List<Predicate> predicates = new ArrayList<>();
9
10             // Exclude incomplete profiles (100% completion required)
11             predicates.add(buildCompleteProfileFilter(root, cb));
12
13             // Create user join for name/email search
14             Join<EmployeeEntity, UserEntity> userJoin =
15                 root.join("user", JoinType.LEFT);
16
17             // Text search: name, email, position
18             if (criteria.searchTerm() != null &&
19                 !criteria.searchTerm().isBlank()) {
20                 String pattern = "%" +
21                     criteria.searchTerm().toLowerCase() + "%";
22
23                 Predicate firstName =
24                     cb.like(cb.lower(userJoin.get("firstName")), pattern);
25                 Predicate lastName =
26                     cb.like(cb.lower(userJoin.get("lastName")), pattern);
27                 Predicate email =
28                     cb.like(cb.lower(userJoin.get("email")), pattern);
29                 Predicate position =
30                     cb.like(cb.lower(root.get("position")), pattern);
31
32                 predicates.add(cb.or(firstName, lastName, email, position))
33                 ;
34             }
35
36             // Skills filter (by UUID)
37             if (criteria.skillIds() != null &&
38                 !criteria.skillIds().isEmpty()) {
39                 Join<EmployeeEntity, EmployeeSkillEntity> skillJoin =
40                     root.join("employeeSkills", JoinType.INNER);
41                 Join<EmployeeSkillEntity, SkillEntity> skillRef =
42                     skillJoin.join("skill", JoinType.INNER);
43                 predicates.add(skillRef.get("id").in(criteria.skillIds()));
44             }

```

```

45      // Location filter (by UUID)
46      if (criteria.locationIds() != null &&
47          !criteria.locationIds().isEmpty()) {
48          Join<EmployeeEntity, LocationEntity> locationJoin =
49              root.join("location", JoinType.INNER);
50          predicates.add(locationJoin.get("id").in(
51              criteria.locationIds()));
52      }
53
54      // Availability filter
55      if (criteria.availability() != null &&
56          !criteria.availability().isEmpty()) {
57          predicates.add(root.get("availability").in(
58              criteria.availability()));
59      }
60
61      // Minimum experience filter
62      if (criteria.minExperience() != null &&
63          criteria.minExperience() > 0) {
64          predicates.add(cb.greaterThanOrEqualTo(
65              root.get("yearsOfExperience"),
66              criteria.minExperience()));
67      }
68
69      return cb.and(predicates.toArray(new Predicate[0]));
70  };
71  }
72
73  private static Predicate buildCompleteProfileFilter(
74      Root<EmployeeEntity> root, CriteriaBuilder cb) {
75
76      // Only 100% complete profiles
77      Predicate hasPosition = cb.isNotNull(root.get("position"));
78      Predicate hasLocation = cb.isNotNull(root.get("location"));
79      Predicate hasSkills = cb.isNotEmpty(root.get("employeeSkills"))
80      ;
81      Predicate hasBio = cb.and(
82          cb.isNotNull(root.get("bio")),
83          cb.notEqual(cb.trim(root.get("bio")), ""));
84      Predicate hasExperience =
85          cb.greaterThan(root.get("yearsOfExperience"), 0);
86
87      return cb.and(hasPosition, hasLocation, hasSkills,
88          hasBio, hasExperience);
89  }

```

Flow-Zusammenfassung: Der Request durchläuft folgende Schichten:

1. **Controller:** Validiert Parameter, mappt DTOs zu Domain-Objekten, delegiert an Service
2. **Service:** Orchestriert Business Logic, nutzt Repository-Ports
3. **DataSource:** Baut JPA Specification, führt Query aus, lädt Top-Skills in Batch
4. **Specification:** Erstellt dynamische WHERE-Klauseln basierend auf Suchkriterien
5. **Mapper:** Konvertiert JPA-Entities zurück zu immutable Domain-Records

A.2.2.2 Frontend: Projektdetail-Feature

Die folgende Feature-Implementierung zeigt das Zusammenspiel von Custom Hook, RTK Query und React-Komponenten am Beispiel der Projektdetailansicht.

Custom Hook (Business Logic Layer)

Listing A.6: useProjectDetail - Custom Hook

```

1 // src/screens/project-details/hooks/use-project-detail.ts
2 import { useGetProjectByIdQuery } from '@store/api/project-api';
3
4 export const useProjectDetail = (projectId: string) => {
5   const {
6     data: project,
7     isLoading,
8     isError,
9     refetch,
10    isFetching
11  } = useGetProjectByIdQuery(projectId, {
12    skip: !projectId, // Skip query if no projectId
13  });
14
15  return {
16    project,
17    isLoading,
18    isError,
19    refetch,
20    isFetching,
21  };
22 };

```

Feature Screen (Presentation Layer)

Listing A.7: ProjectDetailPage - React Component

```

1 // src/screens/project-details/project-details-page.tsx
2 'use client';
3
4 import { useTranslations } from 'next-intl';
5 import { useRouter } from '@libs/i18nNavigation';

```

```

6 import { ProjectDetailHeader } from '../components/project/...';
7 import { ProjectDetailInfo } from '../components/project/...';
8 import { ProjectDetailMembers } from '../components/project/...';
9 import { ProjectDetailTechnologies } from '../components/project/...
  ';
10 import { useProjectDetail } from '../hooks/use-project-detail';
11 import { ErrorState } from "@components/ui/error-state";
12
13 interface ProjectDetailPageProps {
14   readonly projectId: string;
15 }
16
17 export default function ProjectDetailPage(
18   { projectId }: ProjectDetailPageProps
19 ) {
20   const t = useTranslations('projectDetail');
21   const router = useRouter();
22
23   // Custom Hook encapsulates data fetching logic
24   const { project, isLoading, isError, refetch, isFetching } =
25     useProjectDetail(projectId);
26
27   // Error State with retry capability
28   if (isError || (!isLoading && !project)) {
29     return (
30       <ErrorState
31         title={t('error')}
32         description={t('errorDescription')}
33         isRetrying={isFetching}
34         onRetry={refetch}
35         goBack={() => router.push('/dashboard/discover/
           projects')}
36       />
37     );
38   }
39
40   // Main Layout with responsive grid
41   return (
42     <div className="min-h-screen bg-muted/30">
43       <div className="mx-auto max-w-[1440px] px-6 py-8
44         lg:px-16 lg:py-12">
45         <div className="space-y-6">
46           <ProjectDetailHeader
47             project={project}
48             isLoading={isLoading}
49           />
50
51           <div className="grid gap-6 lg:grid-cols-3">

```

```

52         <div className="space-y-6 lg:col-span-2">
53             <ProjectDetailInfo
54                 project={project}
55                 isLoading={isLoading}
56             />
57             <ProjectDetailMembers
58                 project={project}
59                 isLoading={isLoading}
60             />
61         </div>
62
63         <div className="space-y-6">
64             <ProjectDetailTechnologies
65                 project={project}
66                 isLoading={isLoading}
67             />
68         </div>
69     </div>
70 </div>
71 </div>
72 </div>
73 );
74 }

```

Architektur-Pattern: Die Implementierung folgt dem Separation of Concerns-Prinzip:

1. **Custom Hook:** Kapselt Data-Fetching-Logik mit RTK Query, wiederverwendbar
2. **Feature Screen:** Koordiniert UI-Komponenten, handhabt Error/Loading States
3. **Child Components:** ProjectDetailHeader, ProjectDetailInfo, ProjectDetailMembers, ProjectDetailTechnologies – jeweils eigenständige, testbare Komponenten
4. **State Management:** RTK Query Cache sorgt für automatisches Caching, Background-Refetching und optimistische Updates

A.2.2.3 Redux Toolkit Query API Slice

Listing A.8: RTK Query API Slice für Employee-Suche

```

1 // store/api/employee-api.ts
2 export const employeeApi = createApi({
3     reducerPath: 'employeeApi',
4     baseQuery: baseQueryWithLocale,
5     tagTypes: ['Employees', 'EmployeeDetail', 'FilterOptions'],
6     endpoints: (builder) => ({
7         searchEmployees: builder.query<EmployeeSearchResponse,
8                                     EmployeeSearchParams>({
9             query: (params) => {
10                 const searchParams = buildEmployeeSearchParams(params);
11                 const queryString = searchParams.toString();
12

```

```

13         return {
14             url: '/v1/employees/search${queryString ?
15                 '?' + queryString : ''}',
16             method: 'GET',
17         };
18     },
19     providesTags: ['Employees'],
20 },
21
22 getFilterOptions: builder.query<EmployeeFilterOptions, void>({
23     query: () => ({
24         url: '/v1/employees/filter-options',
25         method: 'GET',
26     }),
27     providesTags: ['FilterOptions'],
28 },
29 },
30 });

```

A.2.2.4 Webhook-basierte User-Synchronisation

Listing A.9: Keycloak Webhook Controller

```

1 @RestController
2 @RequestMapping("/v1/webhooks/keycloak")
3 public class KeycloakWebhookController {
4     private final UserSyncService userSyncService;
5
6     @PostMapping
7     public ResponseEntity<Void> handleWebhook(
8         @Valid @RequestBody KeycloakWebhookEventDto event) {
9
10         switch (event.eventType()) {
11             case "CREATE" -> handleCreate(event);
12             case "UPDATE" -> handleUpdate(event);
13             case "DELETE" -> handleDelete(event);
14             default -> { return ResponseEntity.badRequest().build()
15                 ; }
16         }
17         return ResponseEntity.ok().build();
18     }
19
20     private void handleCreate(KeycloakWebhookEventDto event) {
21         userSyncService.createUser(event.toUser());
22     }
23
24     private void handleUpdate(KeycloakWebhookEventDto event) {
25         userSyncService.updateUser(event.userId(), event.toUser());
26     }
27 }

```

```
25     }
26
27     private void handleDelete(KeycloakWebhookEventDto event) {
28         userSyncService.deleteUser(event.userId());
29     }
30 }
```

A.2.2.5 Rollenbasierte Autorisierung

Listing A.10: ProjectManagement Controller mit @PreAuthorize

```
1  @RestController
2  @RequestMapping("/v1/management/projects")
3  @PreAuthorize("hasRole('MANAGER')")
4  public class ProjectManagementController {
5
6      private final ProjectManagementService projectService;
7      private final AuthorizationUtils authorizationUtils;
8
9      @PostMapping
10     public ResponseEntity<ProjectResponseDto> createProject(
11         @Valid @RequestBody CreateProjectRequestDto request,
12         Principal principal) {
13
14         UUID managerId = authorizationUtils.resolveUserId("me",
15             principal);
16         var project = projectService.createProject(request.toDomain
17             (), managerId);
18
19         return ResponseEntity.status(HttpStatus.CREATED)
20             .body(ProjectResponseDto.from(project)
21                 );
22     }
23
24     @GetMapping
25     public ResponseEntity<List<ProjectResponseDto>> getMyProjects(
26         Principal principal) {
27         UUID managerId = authorizationUtils.resolveUserId("me",
28             principal);
29         var projects = projectService.getProjectsByManager(
30             managerId);
31
32         return ResponseEntity.ok(
33             projects.stream()
34                 .map(ProjectResponseDto::from)
35                 .toList()
36         );
37     }
38 }
```

A.3 Diagramme

A.3.1 Sequenzdiagramme

A.3.1.1 OAuth2 Authentifizierung und Autorisierung

Abbildung A.1 zeigt den Authentifizierungs- und Autorisierungsablauf mit Token Introspection:

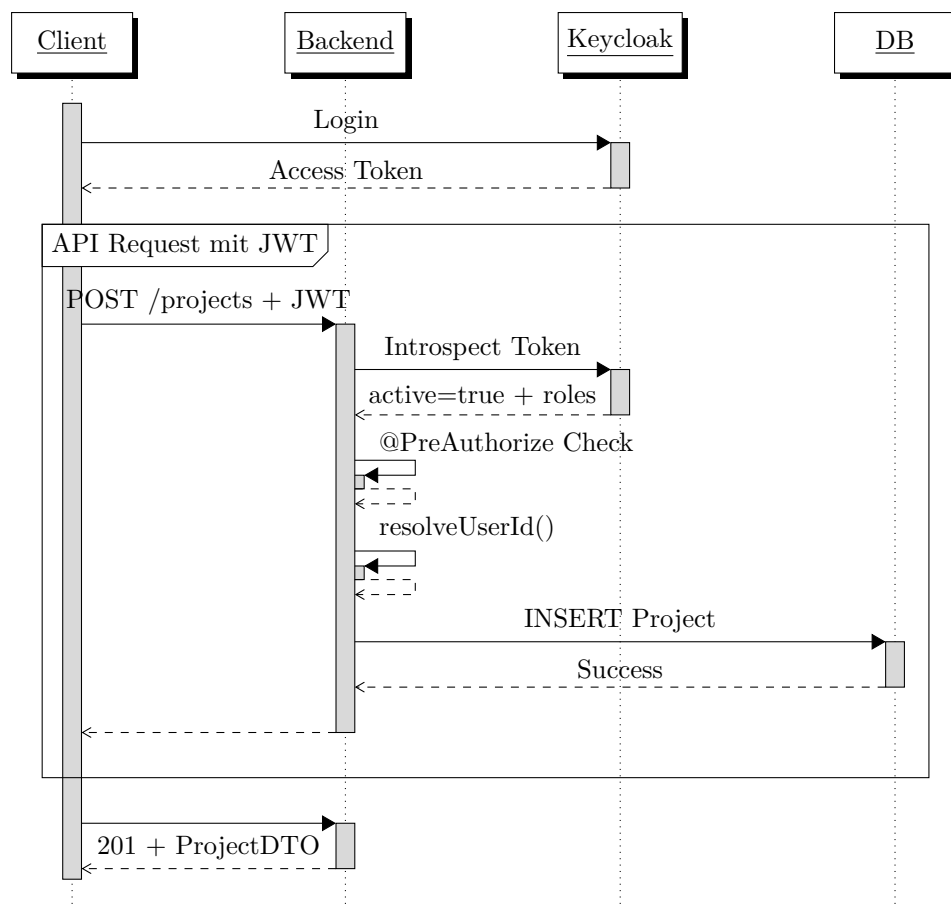


Abbildung A.1: OAuth2 Token Introspection Flow mit rollenbasierter Autorisierung

Der Ablauf umfasst folgende Schritte:

1. **Login:** Client authentifiziert sich bei Keycloak und erhält JWT Access Token
2. **API-Request:** Client sendet Request mit JWT im Authorization-Header (Bearer <token>)
3. **Token Introspection:** Backend validiert Token bei Keycloak über /protocol/openid-connect/token/introspect – Keycloak prüft Signatur, Gültigkeit und gibt `active=true` plus Rollen zurück
4. **Rollen-Extraktion:** Spring Security extrahiert Rollen aus Introspection-Response (`realm_access.roles`)

5. **@PreAuthorize-Check:** Annotation prüft erforderliche MANAGER-Rolle vor Method-Ausführung
6. **User-Auflösung:** `AuthorizationUtils.resolveUserId("me")` extrahiert User-ID aus Principal (Subject-Claim)
7. **Business Logic:** Service-Layer führt Geschäftslogik aus, persistiert Projekt mit Manager-Referenz
8. **Response:** Backend gibt 201 Created mit ProjectDTO zurück

A.3.1.2 Mitarbeitersuche mit dynamischer Filterung

Abbildung A.2 zeigt den vollständigen Request-Flow der Mitarbeitersuche mit JPA Specifications:

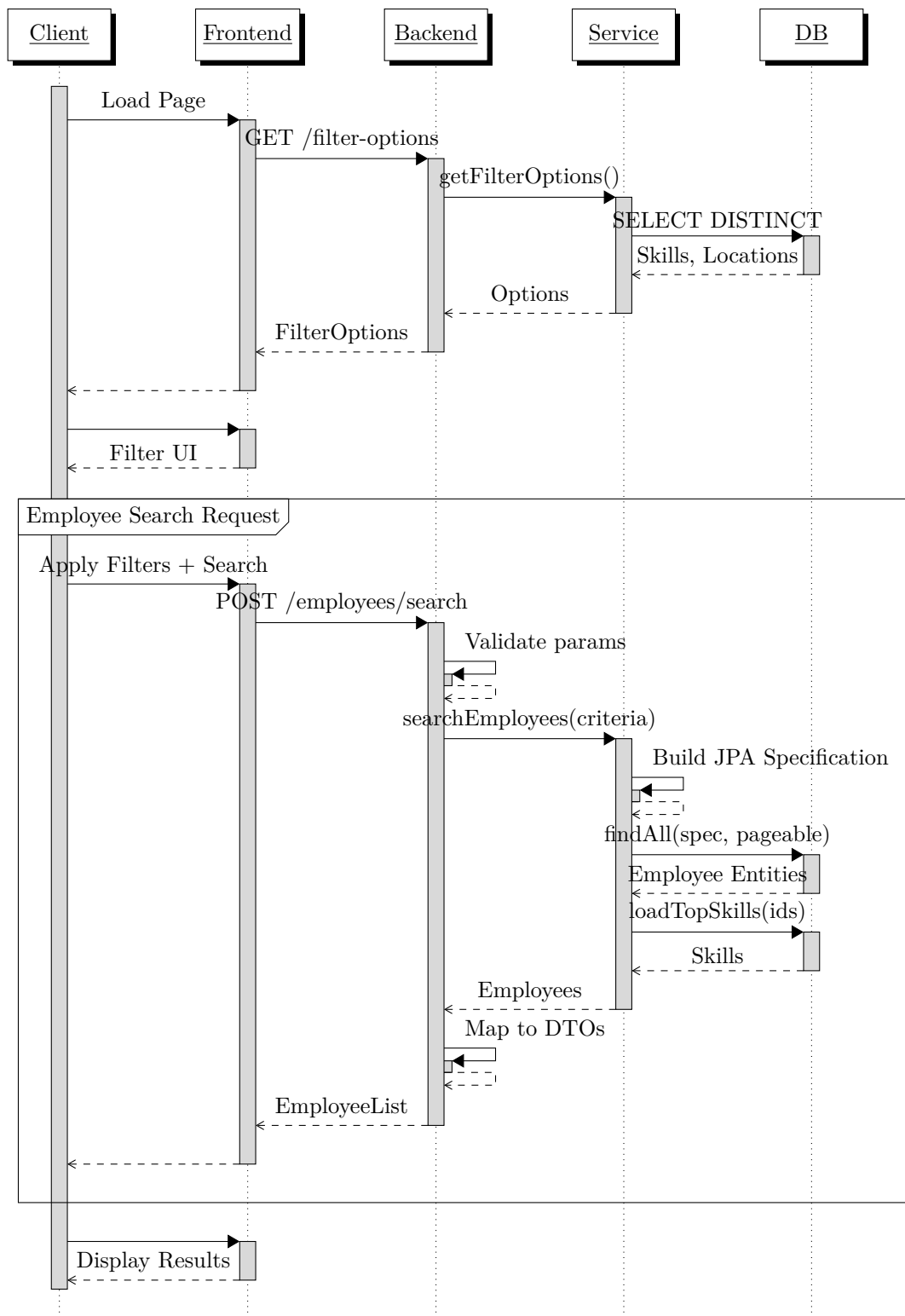


Abbildung A.2: Mitarbeitersuche: Request-Flow mit JPA Specification Pattern

Der Suchablauf demonstriert das Zusammenspiel aller Architekturschichten:

1. **Filter-Optionen laden:** Frontend ruft `/v1/employees/filter-options` auf, Backend aggregiert verfügbare Skills, Kategorien und Standorte aus DB
2. **Client-Filterung:** User wählt Filter (Skills, Standort, Verfügbarkeit, Mindest-Erfahrung)
3. **Search Request:** Frontend sendet POST mit `EmployeeSearchCriteria`

4. **Parameter Validation:** Backend validiert Eingaben via `@Valid`
5. **Dynamic Query Building:** `EmployeeSpecification` erstellt JPA Criteria API Query mit dynamischen WHERE-Klauseln
6. **Datenbankabfrage:** Repository führt `findAll(spec, pageable)` mit Pagination aus
7. **Batch Loading:** Top-Skills werden in einem Query geladen (N+1-Problem-Vermeidung)
8. **DTO Mapping:** Entities werden zu `EmployeeSearchResultDto` gemappt
9. **Response:** Frontend zeigt Ergebnisse im responsiven Grid

Die vollständige Implementierung (Controller, Service, DataSource, Specification) ist in Anhang A.2 bis A.5 dokumentiert.

A.3.1.3 Skill-Verwaltung (CRUD-Operationen)

Abbildung A.3 zeigt den Skill-CRUD-Workflow mit RTK Query und optimistischen Updates:

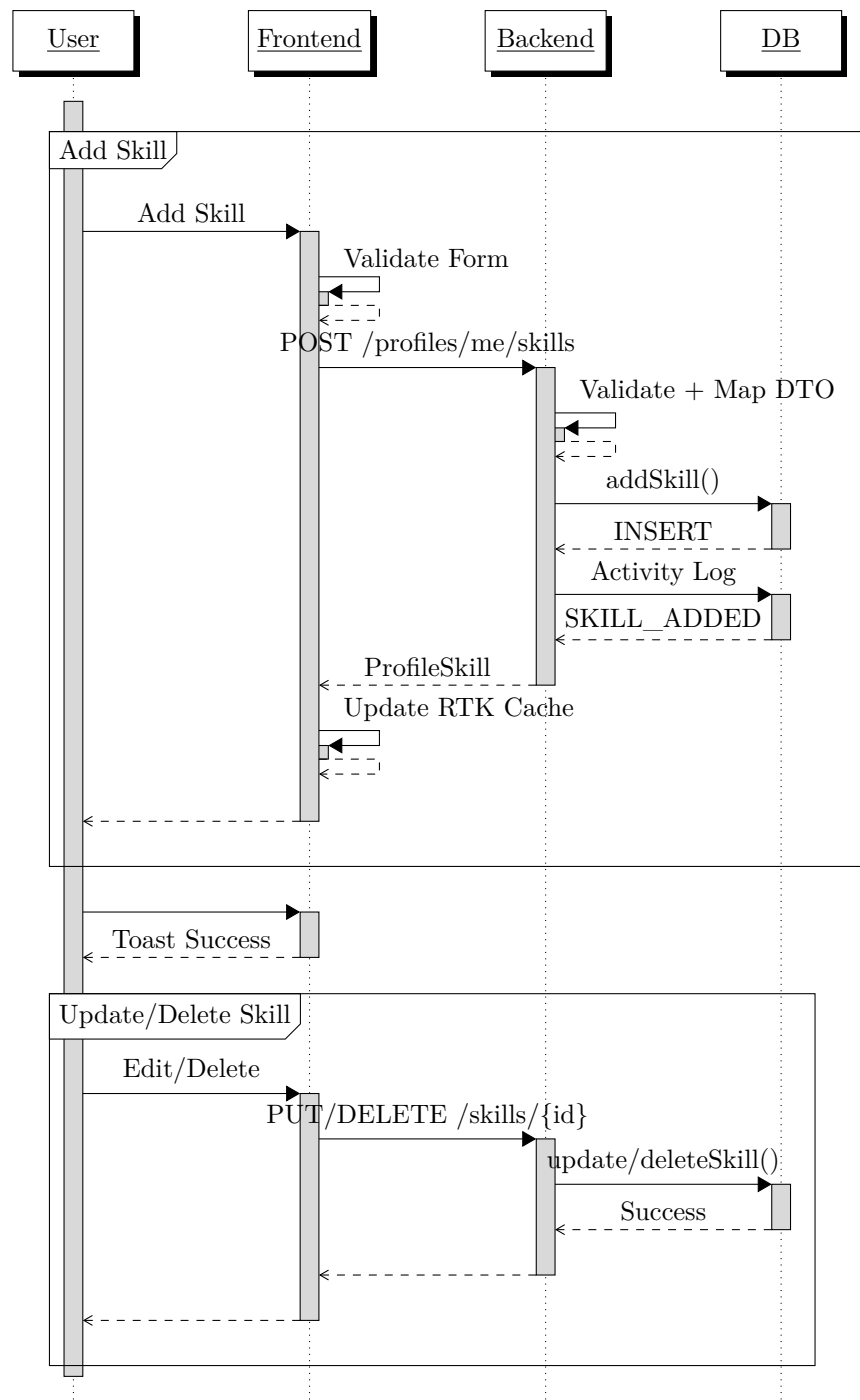


Abbildung A.3: Skill-Verwaltung: CRUD-Workflow mit RTK Query Cache-Management

Der Skill-Management-Workflow demonstriert moderne Frontend-Patterns:

1. **Validation:** Frontend validiert Eingaben (Score 0–100, Pflichtfelder), Backend prüft via `@Valid`
2. **CRUD-Operationen:** POST (`addSkill`), PUT (`updateSkill`), DELETE (`deleteSkill`) mit transaktionaler DB-Persistierung
3. **Activity Logging:** Jede Änderung erstellt Audit-Log-Eintrag (`SKILL_ADDED`, `SKILL_UPDATED`, `SKILL_DELETED`)
4. **Cache Management:** RTK Query aktualisiert Cache nach Server-Response und

invalidiert betroffene Queries (`providesTags: ['Profile']`)

5. **Error Handling:** Bei Fehler wird Toast-Notification angezeigt

Die Implementierung nutzt Custom Hooks (`useSkillManagement`) zur Kapselung der Geschäftslogik.

A.3.2 Aktivitätsdiagramme

A.3.2.1 Projekterstellung mit Mitarbeiterauswahl

Abbildung A.4 zeigt den dreispaltigen Workflow zur Projekterstellung mit integrierter Mitarbeiterauswahl:

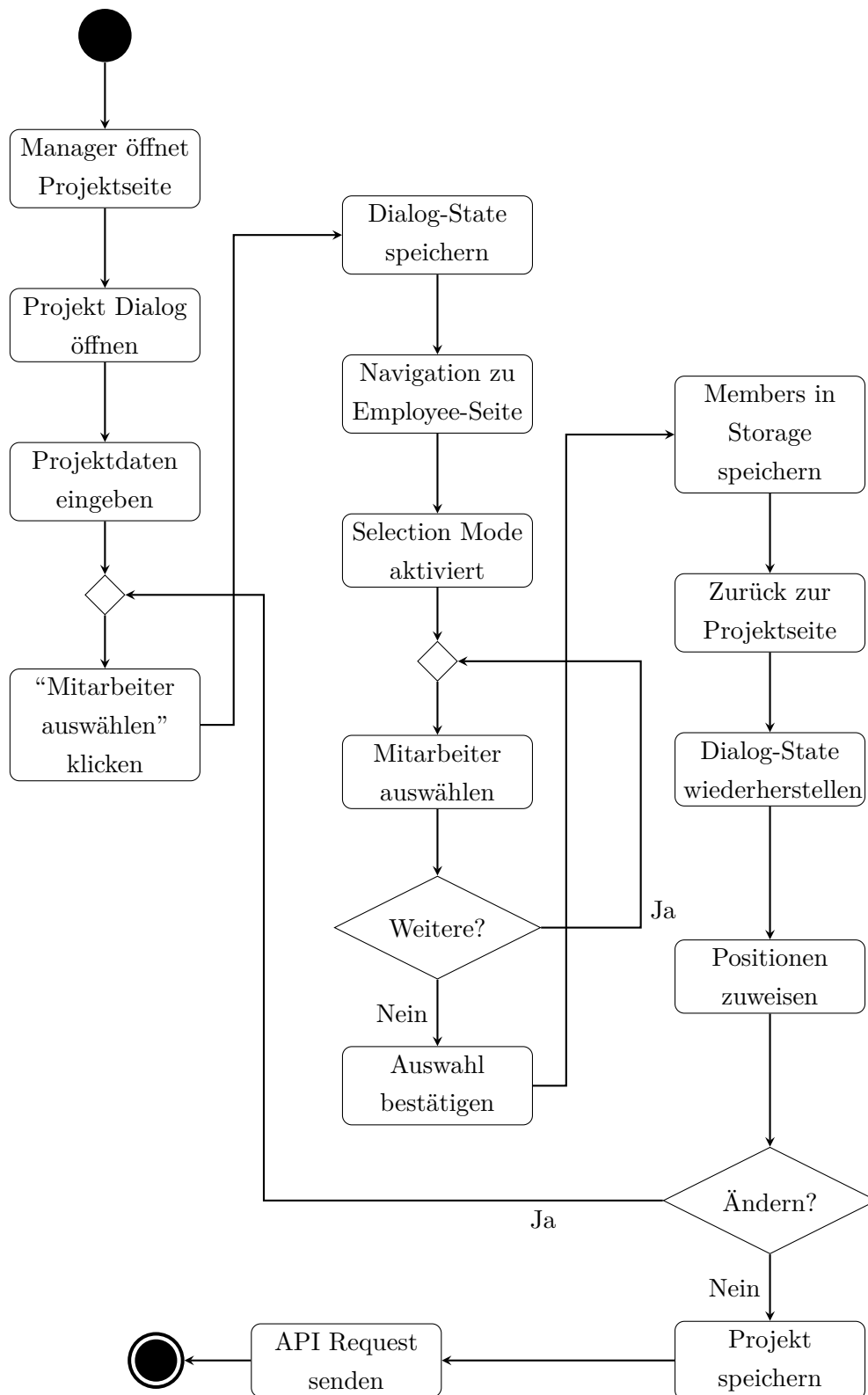


Abbildung A.4: Aktivitätsdiagramm: Projekterstellung mit Mitarbeiterauswahl

A.3.2.2 Mitarbeitersuche mit Multi-Filter

Abbildung A.5 zeigt den Ablauf der Mitarbeitersuche mit dynamischer Filter-Anwendung:

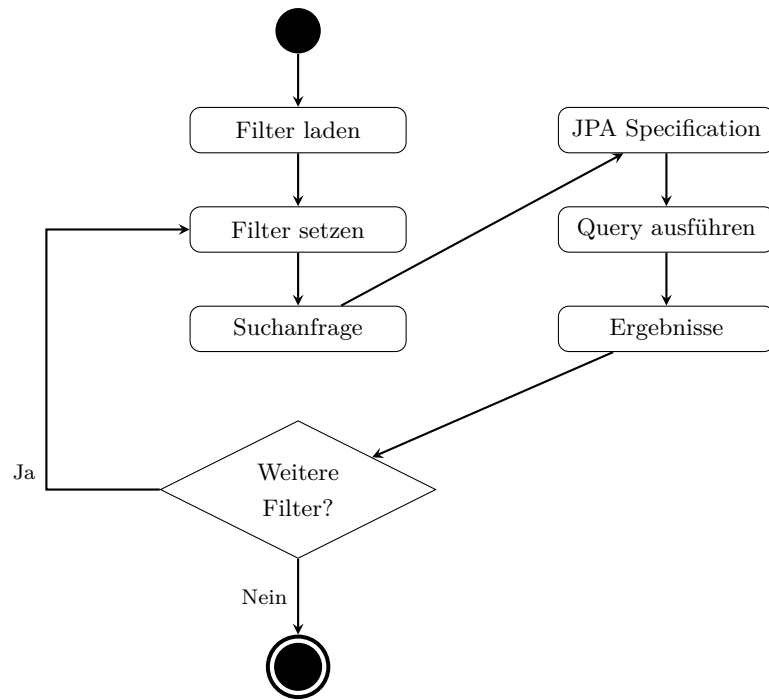


Abbildung A.5: Aktivitätsdiagramm: Mitarbeitersuche mit dynamischen Filtern

Die Aktivitätsdiagramme zeigen den Ablauf komplexer User-Workflows mit JPA Specification Pattern (siehe Kapitel 3.3.1).

A.3.3 Klassendiagramm

A.3.3.1 Domain Model

Abbildung A.6 zeigt die Kernentitäten der Domain-Schicht als UML-Klassendiagramm mit immutable Java Records:

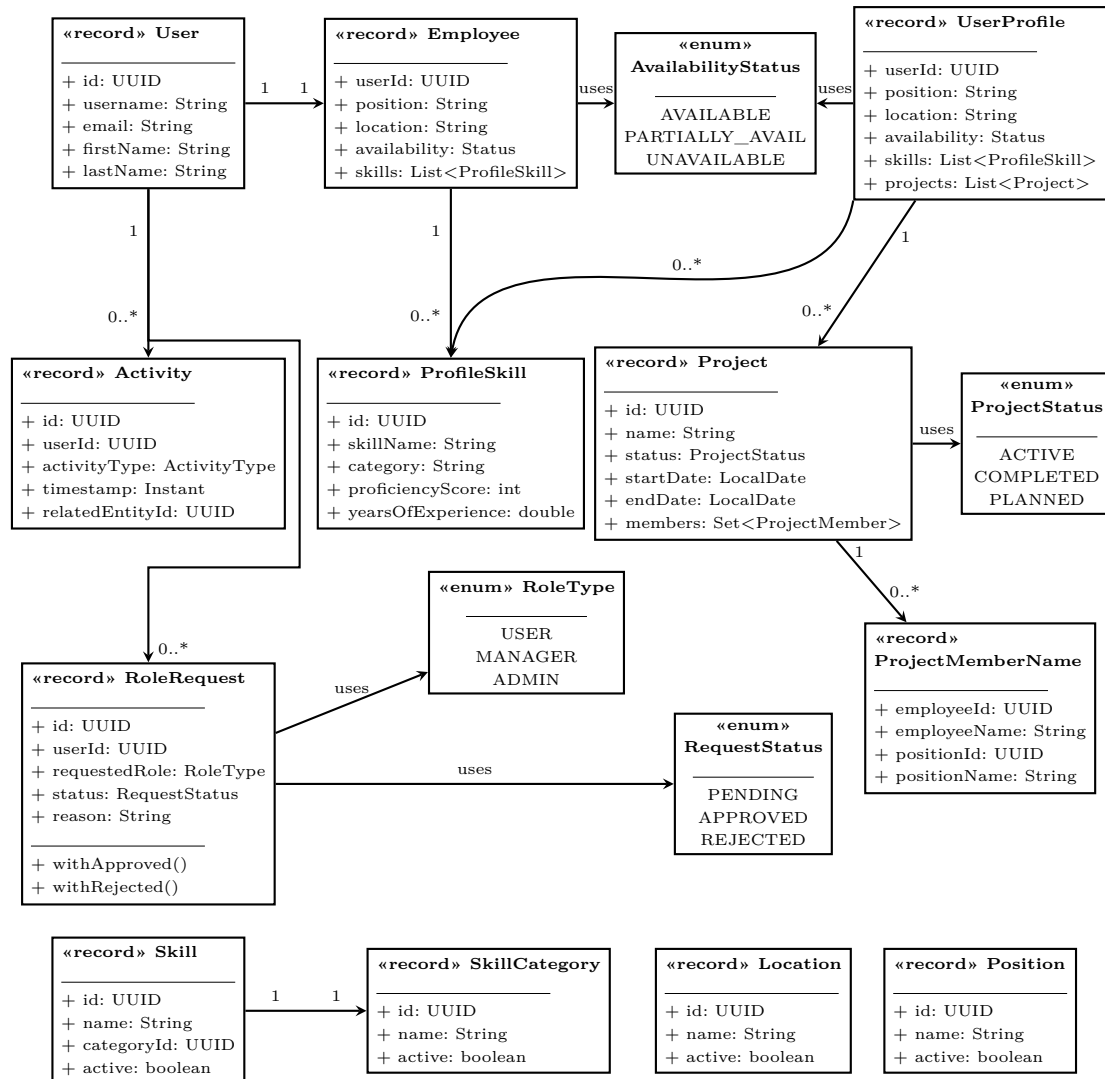


Abbildung A.6: Skill Management System: Domain Model - Kernentitäten

Das Klassendiagramm umfasst die gesamte Domain-Schicht mit zwölf Records (*User*, *Employee*, *UserProfile*, *Activity*, *ProfileSkill*, *Project*, *ProjectMemberName*, *RoleRequest*, *Skill*, *SkillCategory*, *Location*, *Position*) sowie vier Enums (*AvailabilityStatus*, *ProjectStatus*, *RoleType*, *RequestStatus*). Die wichtigsten Relationen sind u. a. **User 1:1 Employee**, **Employee 1:0..* ProfileSkill**, **Project 1:0..* ProjectMemberName** sowie weitere Verknüpfungen zwischen Skills, Kategorien, Position und Location.

Alle Domain-Objekte sind als immutable Java-Records implementiert; Enums werden über *uses*-Beziehungen eingebunden und stellen typsichere Status- und Rollenwerte bereit. *RoleRequest* ergänzt dies um Methoden für Statuswechsel (z. B. *withApproved()*, *withRejected()*).

Die Struktur folgt der Hexagonal Architecture und trennt Domain-Logik klar von Infrastrukturkomponenten (siehe Kapitel 3.3.1).

A.3.4 Zustandsdiagramm

A.3.4.1 Rollen-anfrage-Workflow

Abbildung A.7 zeigt den Zustandsübergang für Self-Service Rollen-anfragen:

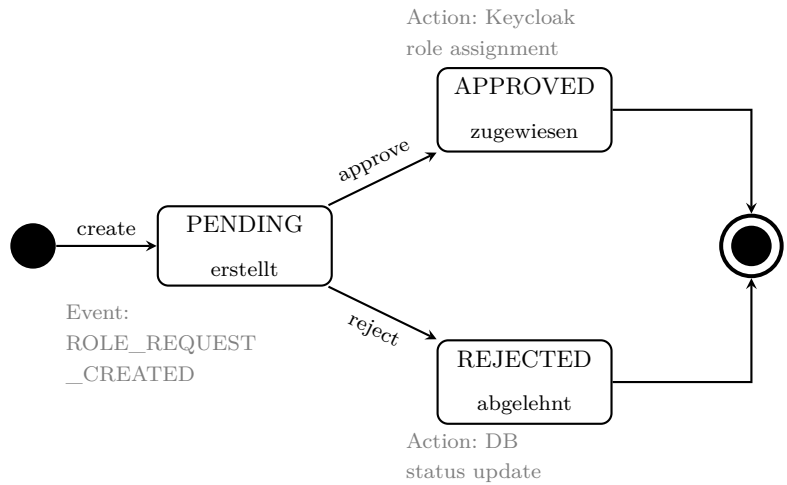


Abbildung A.7: Zustandsdiagramm: Rollen-anfrage-Workflow (Self-Service Pattern)

Der Rollen-anfrage-Workflow ermöglicht Self-Service-Rollenverwaltung:

- 1. **PENDING**: User erstellt Anfrage für MANAGER-Rolle (V14 Migration)
- 2. **APPROVED**: Admin genehmigt → Backend weist Rolle in Keycloak zu → Status-Update in DB → Activity-Log-Eintrag
- 3. **REJECTED**: Admin lehnt ab → Status-Update in DB → Activity-Log-Eintrag

Das Zustandsdiagramm dokumentiert den Self-Service-Ansatz für Rechteverwaltung (siehe Kapitel 3.4.3).

A.4 Qualitätssicherung & CI/CD

A.4.1 Code-Qualitätsmetriken

Metrik	Backend	Frontend	Bewertung
Lines of Code	7.800+	10.500+	Vollständige Implementierung
Maintainability Rating	A	A	Hohe Wartbarkeit, klar strukturiert
Security Rating	A	A	Keine Security Hotspots
Reliability Rating	A	A	Keine kritischen Bugs
Test Coverage	0%	0%	Keine automatisierten Tests
Duplications	< 1%	< 2%	Minimale Code-Redundanz

Tabelle A.6: SonarQube Code-Qualitätsmetriken (Stand: Build #15)

A.4.2 CI/CD und Quality Assurance

A.4.2.1 Commit-Historie und Conventional Commits



Abbildung A.8: Commit-Historie mit Conventional Commits und Commit Lint

Abbildung A.8 zeigt die strukturierte Commit-Historie nach Conventional Commits-Standard mit Commit Lint-Validierung. Die Commit-Messages folgen dem Format `type(scope): description` und ermöglichen automatisches Changelog-Generation und semantische Versionierung.

A.4.2.2 SonarQube Code-Analyse

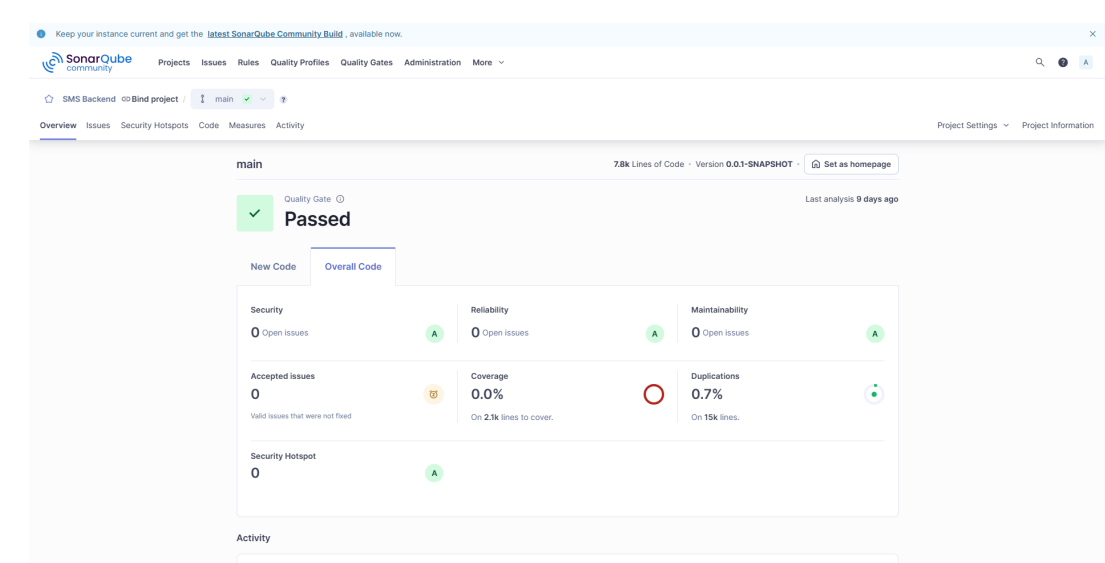


Abbildung A.9: SonarQube Analyse – Backend (SMS Backend, Branch: main)

Backend Quality Gate Abbildung A.9 zeigt die SonarQube-Analyse des Backend-Projekts mit Quality Gate *Passed*. Die Metriken umfassen 7.800+ Lines of Code, 0 Open

Issues in allen Kategorien (Security, Reliability, Maintainability jeweils Rating A), 0.7% Duplications und 15k Lines to Cover (bei 0% Test Coverage).

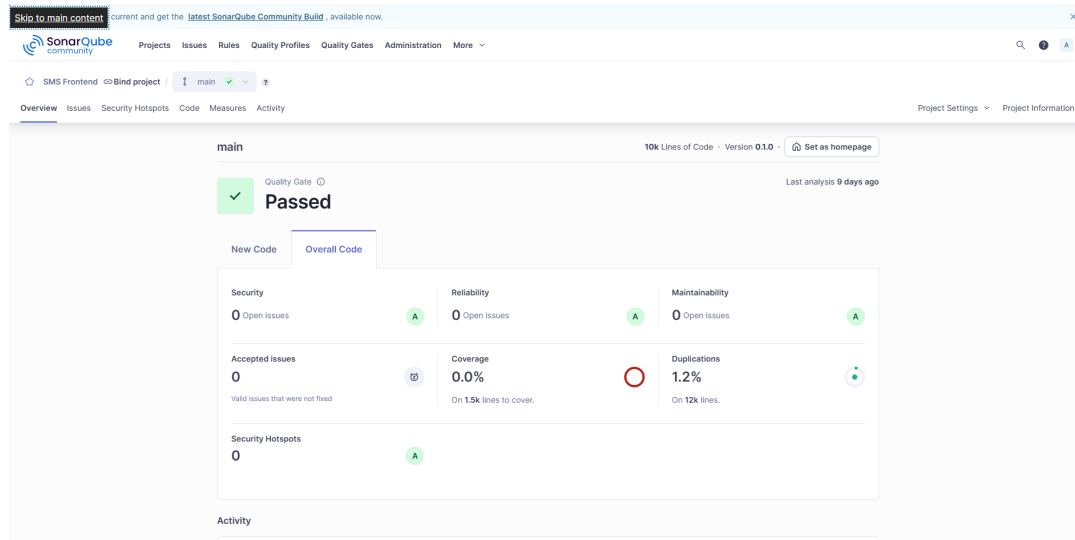


Abbildung A.10: SonarQube Analyse – Frontend (SMS Frontend, Branch: main)

Frontend Quality Gate Abbildung A.10 zeigt die SonarQube-Analyse des Frontend-Projekts mit Quality Gate *Passed*. Die Metriken umfassen 10.500+ Lines of Code, 0 Open Issues (Security/Reliability/Maintainability jeweils Rating A), 1.2% Duplications und 12k Lines to Cover.

A.4.2.3 Jenkins CI/CD Pipeline

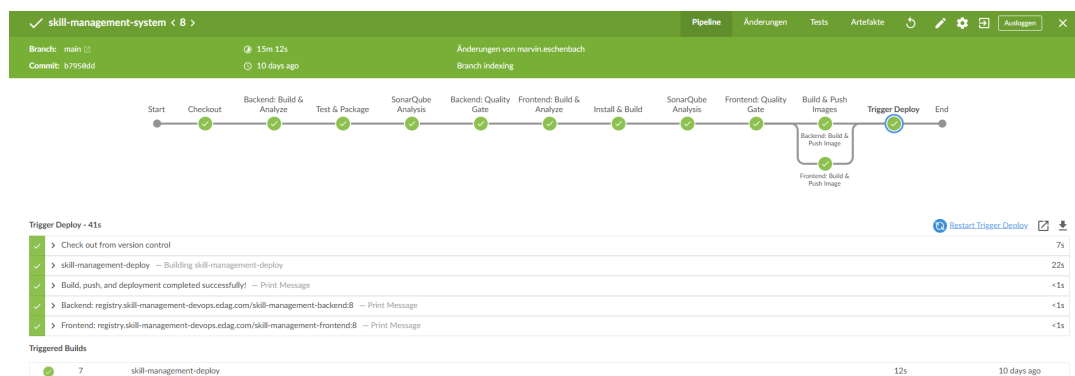


Abbildung A.11: Jenkins Declarative Pipeline – Build #7 (Trigger Deploy Stage)

Abbildung A.11 zeigt die vollständige Jenkins Pipeline-Ausführung mit allen Stages: Checkout (7s), Backend Build & Analyze (22s), Frontend Build & Analyze (Build & Push Images parallel: Backend 12s, Frontend <1s), Trigger Deploy (41s). Die Gesamtlaufzeit betrug

15 Minuten 12 Sekunden. Die Pipeline-Visualisierung zeigt den erfolgreichen Deployment-Prozess vom Branch `main` mit automatischer Image-Erstellung und Kubernetes-Deployment.

A.4.2.4 Jenkinsfile Deklaration

Listing A.11: Jenkins Declarative Pipeline – Jenkinsfile

```

1 pipeline {
2   agent none
3   options {
4     disableConcurrentBuilds()
5     timeout(time: 1, unit: 'HOURS')
6   }
7   environment {
8     REGISTRY_EXTERNAL = 'registry.skill-management-devops.edag.com'
9     BACKEND_IMAGE_EXTERNAL = "${REGISTRY_EXTERNAL}/skill-
10      management-backend"
11     FRONTEND_IMAGE_EXTERNAL = "${REGISTRY_EXTERNAL}/skill-
12      management-frontend"
13   }
14   stages {
15     stage('Checkout') {
16       agent { label 'universal' }
17       steps { checkout scm }
18     }
19     stage('Backend: Build & Analyze') {
20       agent { label 'backend' }
21       stages {
22         stage('Test & Package') {
23           steps {
24             container('maven') {
25               sh '''#!/bin/bash
26                 cd backend
27                 ./mvnw -B clean test package
28               '''
29             }
30           }
31         }
32         stage('SonarQube Analysis') {
33           steps {
34             container('maven') {
35               withSonarQubeEnv('SonarQube') {
36                 sh '''cd backend
37                   ./mvnw sonar:sonar \\
38                     -Dsonar.projectKey=sms-backend
39                 '''
40             }
41           }
42         }
43       }
44     }
45   }
46 }

```

```

41     }
42   }
43 }
44 stage('Backend: Quality Gate') {
45   agent { label 'universal' }
46   steps {
47     timeout(time: 10, unit: 'MINUTES') {
48       waitForQualityGate abortPipeline: true
49     }
50   }
51 }
52 // Frontend stages analog zu Backend
53 stage('Build & Push Images') {
54   when { branch 'main' }
55   parallel {
56     stage('Backend: Build & Push Image') {
57       agent { label 'backend' }
58       steps {
59         container('docker') {
60           script {
61             docker.withRegistry(
62               "https://\${REGISTRY_EXTERNAL}", 'docker-registry
63               ') {
64               dir('backend') {
65                 def img = docker.build(
66                   "\${BACKEND_IMAGE_EXTERNAL}:\${BUILD_NUMBER}"
67                 )
68                 img.push()
69                 img.push('latest')
70               }
71             }
72           }
73         }
74         // Frontend parallel
75       }
76     }
77   stage('Trigger Deploy') {
78     when { branch 'main' }
79     agent { label 'universal' }
80     steps {
81       script { build job: 'skill-management-deploy' }
82     }
83   }
84 }
85 }

```

Die Jenkinsfile-Konfiguration (Listing A.11) definiert eine mehrstufige Pipeline mit Kubernetes-Agents für unterschiedliche Build-Environments (Backend: Maven, Frontend: Node.js, Docker: Image-Builds). Quality Gates stoppen die Pipeline bei SonarQube-Violations. Branch `main` triggert automatisch Docker-Image-Builds und Deployment.

A.4.2.5 Lighthouse Performance Audit

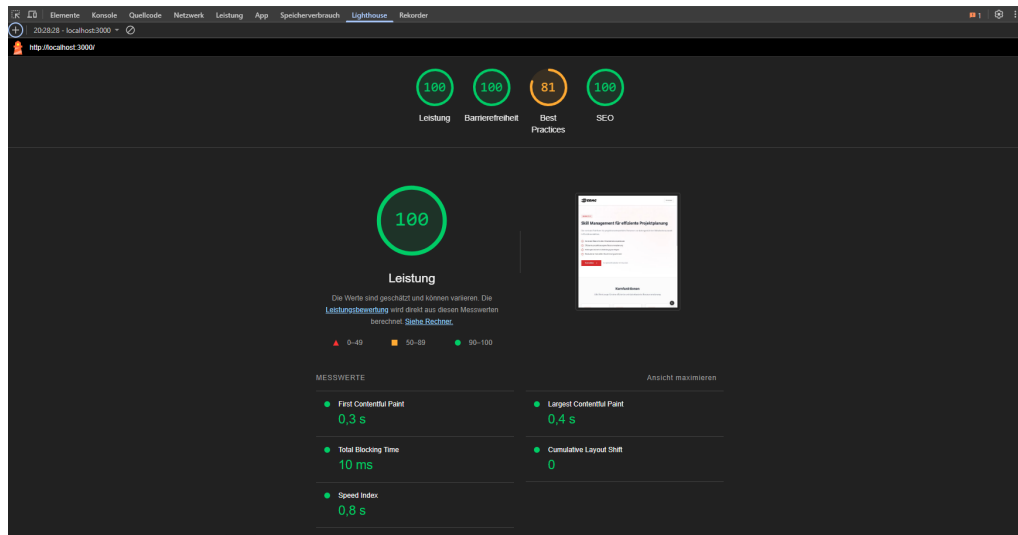


Abbildung A.12: Google Lighthouse Audit – Welcome Screen (Performance: 100, Best Practices: 81)

Abbildung A.12 zeigt das Lighthouse-Audit[49] der Welcome-Seite mit exzellenten Werten: Performance 100/100 (First Contentful Paint 0.3s, Largest Contentful Paint 0.4s, Speed Index 0.8s), Accessibility 100/100, Best Practices 81/100, SEO 100/100. Die Messwerte bestätigen die Performance-Anforderungen aus Kapitel 1.3.2.

A.5 Deployment & Konfiguration

A.5.1 ER-Datenmodell

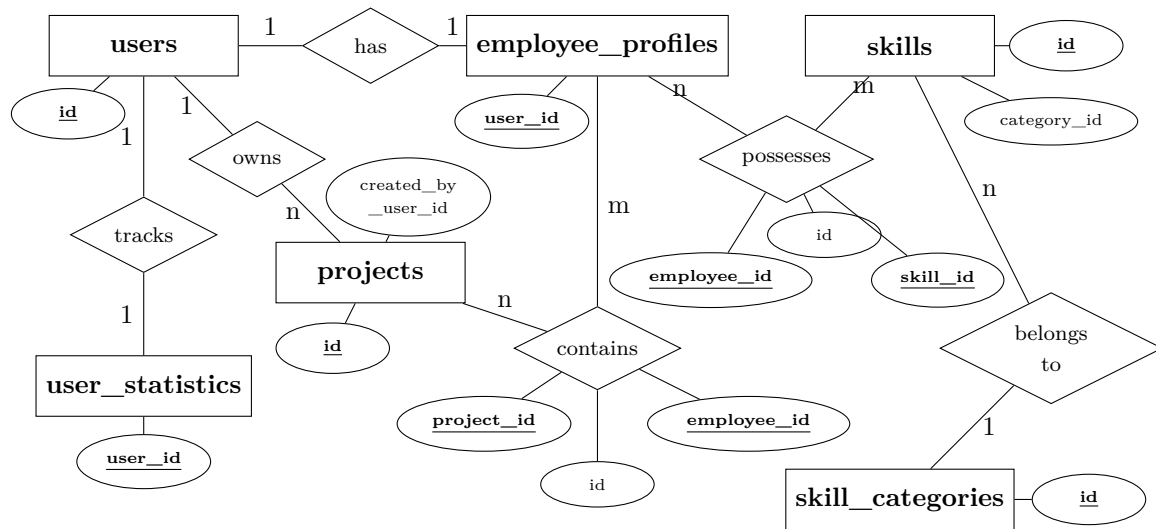


Abbildung A.13: Entity-Relationship-Diagramm nach UML-Standard (Kernentitäten)

A.5.1.1 Datenbankmodell

Abbildung A.14 zeigt die tatsächliche Datenbankstruktur mit allen 15 Tabellen (14 Anwendungstabellen + `flyway_schema_history`), ihren Spalten, Datentypen und Constraints.

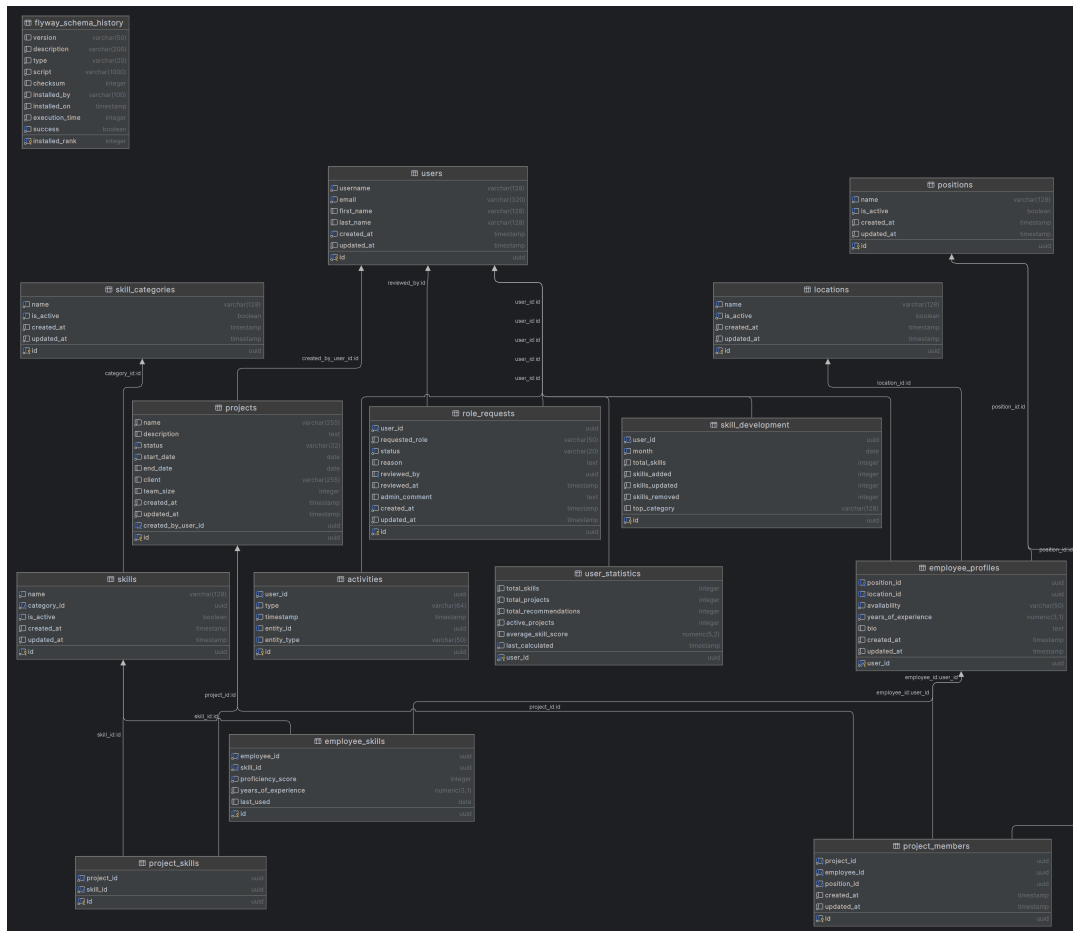


Abbildung A.14: Datenbank-Schema: Vollständige Tabellenstruktur mit 15 Tabellen

A.5.2 Datenbank-Migrationen

Die 15 Flyway-Migrationsskripte dokumentieren die vollständige Datenbank-Evolution von Basis-Tabellen über automatisierte Trigger bis zu umfassenden Analytics. Die versionierte Schema-Migration gewährleistet reproduzierbare Deployments und nachvollziehbare Änderungen.

A.5.2.1 Migrations-Übersicht

Nr.	Kategorie	Beschreibung	LOC
V1	Schema	users, positions, locations, skill_categories, skills	92
V2	Schema	employee_profiles, employee_skills (Junction)	56
V3	Schema	projects, project_skills, project_positions	143
V4	Analytics	user_statistics, skill_development, activities	102
V5	Trigger	Automatische updated_at-Timestamps (7 Tabellen)	48
V6	Trigger	Echtzeit-Aggregation: total_skills, avg_score, project_counts	131
V7	Trigger	Audit-Trail: SKILL_ADDED, PROJECT_CREATED, PROFILE_UPDATED	134
V8	Trigger	Monatliche Skill-Entwicklung Snapshots	177
V9	Daten	200+ Skills, 55+ Positionen, 30 Standorte	379
V10	Trigger	Auto-Erstellung employee_profiles bei User-Insert	47
V11	Refactoring	project_members n:m-Tabelle (eliminiert Redundanz)	278
V12	Trigger	Automatische Team-Size-Berechnung	36
V13	Refactoring	Entfernung team_size CHECK-Constraint	7
V14	Schema	role_requests (Self-Service Rollenanfragen)	38
V15	Trigger	30+ Event-Typen: ROLE_REQUEST_APPROVED, PROJECT_MEMBER_ADDED	358
Gesamt	15 Dateien	Vollständig versioniertes Datenbankschema	2.026

Tabelle A.7: Vollständige Übersicht aller Flyway-Datenbank-Migrationen

Beispiel-Listings Die folgenden Auszüge zeigen exemplarisch die Struktur der ersten zwei Migrationen. Alle weiteren 13 Migrationen folgen dem gleichen Schema mit vollständiger Dokumentation durch SQL-Kommentare.

V1: Basis-Tabellen (Auszug)

```

1  -- V1__create_base_tables.sql
2  CREATE EXTENSION IF NOT EXISTS pg_trgm; -- Full-text search
3
4  CREATE TABLE users (
5      id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
6      username VARCHAR(128) NOT NULL UNIQUE,
7      email VARCHAR(320) NOT NULL UNIQUE,
8      first_name VARCHAR(128),
9      last_name VARCHAR(128),
10     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
11 );
12
13 CREATE TABLE skills (
14     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
15     name VARCHAR(128) NOT NULL UNIQUE,
16     category_id UUID NOT NULL,
17     CONSTRAINT fk_skills_category FOREIGN KEY (category_id)
18         REFERENCES skill_categories(id) ON DELETE RESTRICT
19 );

```

V2: Employee Skills Junction-Tabelle (Auszug)

```

1  -- V2__create_employee_profiles.sql
2  CREATE TABLE employee_skills (
3      id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
4      employee_id UUID NOT NULL,
5      skill_id UUID NOT NULL,
6      proficiency_score INTEGER NOT NULL,
7      CONSTRAINT fk_employee_skill_employee FOREIGN KEY (employee_id)
8          REFERENCES employee_profiles(user_id) ON DELETE CASCADE,
9      CONSTRAINT chk_proficiency_score CHECK
10         (proficiency_score >= 0 AND proficiency_score <= 100)
11 );

```

Repository-Pfad: Alle 15 vollständigen SQL-Migrationsskripte (2.026 LOC) sind verfügbar unter:

backend/src/main/resources/db/migration/V*.sql

A.5.3 Kubernetes Ingress-Konfiguration

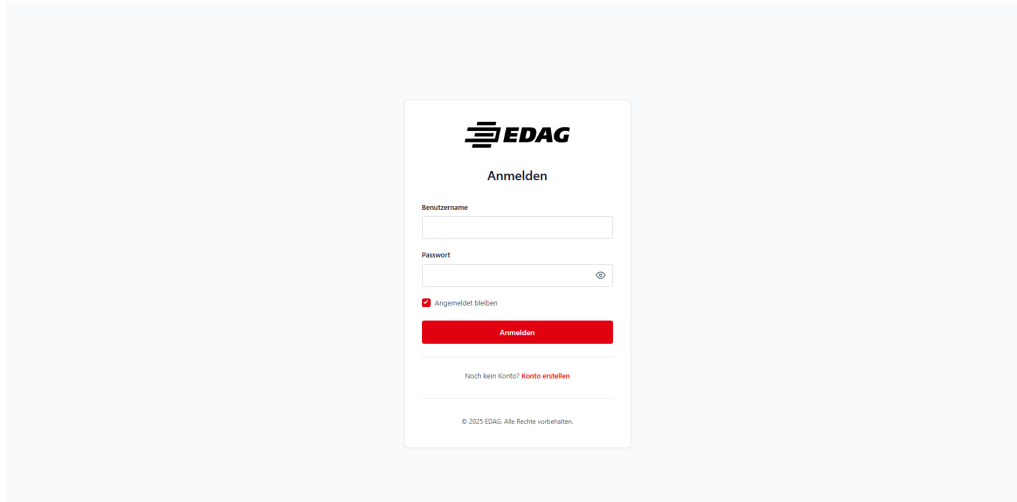
Hostname	Service	Beschreibung
keycloak.dev.local	keycloak:8080	Identity Provider (Dev)
skill-management.edag.com	frontend:3000	Next.js Frontend (Prod)
api.skill-management.edag.com	backend:8080	Spring Boot Backend (Prod)
keycloak.skill-management.edag.com	keycloak:8080	Keycloak IAM (Prod)
jenkins.devops.local	jenkins:8080	CI/CD Pipeline
sonarqube.devops.local	sonarqube:9000	Code-Qualität
registry.devops.local	docker-registry:5000	Docker Registry

Tabelle A.8: Traefik Ingress-Konfiguration für alle Environments

A.6 Dokumentation

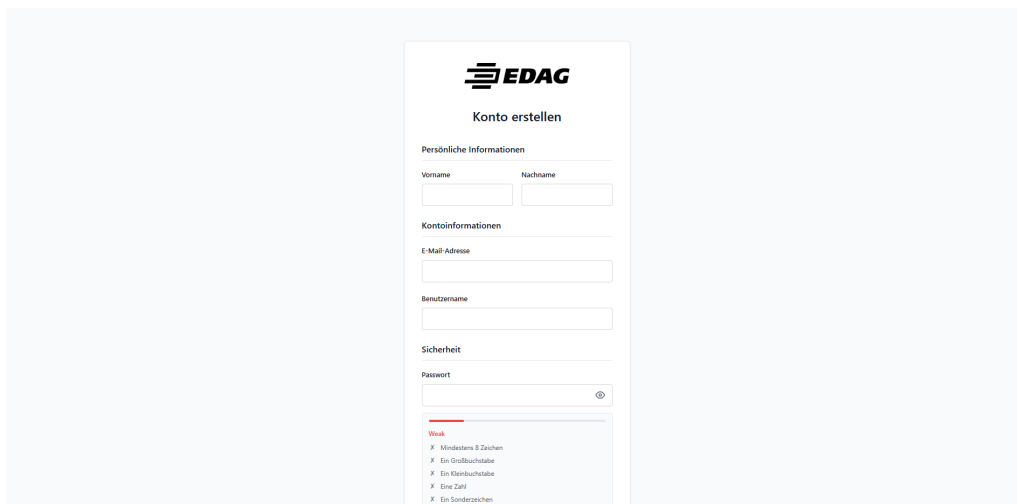
A.6.1 UI-Screenshots

A.6.1.1 Authentifizierung und Onboarding



The screenshot shows the EDAG login page. At the top is the EDAG logo. Below it is the title "Anmelden". There are two input fields: "Benutzername" and "Passwort". The "Passwort" field has a toggle icon for visibility. Below the password field is a checkbox labeled "Angemeldet bleiben" which is checked. A red button labeled "Anmelden" is below the checkbox. At the bottom, there is a link "Noch kein Konto? Konto erstellen" and a footer "© 2025 EDAG. Alle Rechte vorbehalten."

Abbildung A.15: Login-Seite mit Keycloak OAuth2/OIDC-Integration



The screenshot shows the EDAG account creation page. At the top is the EDAG logo. Below it is the title "Konto erstellen". There are three sections: "Persönliche Informationen" with "Vorname" and "Nachname" fields; "Kontoinformationen" with "E-Mail-Adresse" and "Benutzername" fields; and "Sicherheit" with a "Passwort" field and a "Wirk" (Validity) section. The "Wirk" section lists requirements: "Mindestens 8 Zeichen", "Ein Großbuchstabe", "Ein Kleinbuchstabe", "Eine Zahl", and "Ein Sonderzeichen".

Abbildung A.16: Keycloak-Authentifizierungsmaske

E-Mail-Adresse

Benutzername

Sicherheit

Passwort

Weak

X

Mindestens 8 Zeichen

X

Ein Großbuchstabe

X

Ein Kleinbuchstabe

X

Eine Zahl

X

Ein Sonderzeichen

Passwort bestätigen

Konto erstellen

Bereits ein Konto? [Anmelden](#)

© 2025 EDAG. Alle Rechte vorbehalten.

Abbildung A.17: Login-Erfolgsmeldung und Weiterleitung

EDAG

Anmelden

MOBILITY IT

Skill Management für effiziente Projektplanung

Die zentrale Plattform für projektverantwortliche Personen zur datengestützten Mitarbeiterauswahl in Kundenprojekten.

Zentrale Übersicht aller Mitarbeiterkompetenzen

Effiziente projektbezogene Ressourcenplanung

Datengestützte Entscheidungsgrundlagen

Reduzierter manueller Abstimmungsaufwand

Anmelden

Für EDAG Mitarbeiter mit Keycloak

Skills 1.000+

Teams 50+

Skill Matrix

Intelligente Ressourcenzuweisung

User 500+

Abbildung A.18: Welcome-Seite: Erste Schritte nach Registrierung

EDAG

Anmelden

Kernfunktionen

Alle Werkzeuge für eine effiziente und datenbasierte Ressourcenplanung

Kompetenz-Tracking

Erfassen und verwalten Sie alle Fähigkeiten und Kompetenzen Ihrer Mitarbeiter an einem zentralen Ort.

Team-Zusammenarbeit

Fördern Sie die Zusammenarbeit zwischen Teams durch transparente Kompetenzübersichten.

Karriereentwicklung

Unterstützen Sie die berufliche Entwicklung durch gezielte Kompetenzanalysen und -empfehlungen.

Abbildung A.19: Welcome-Seite: Profilkonfiguration

Seite 61

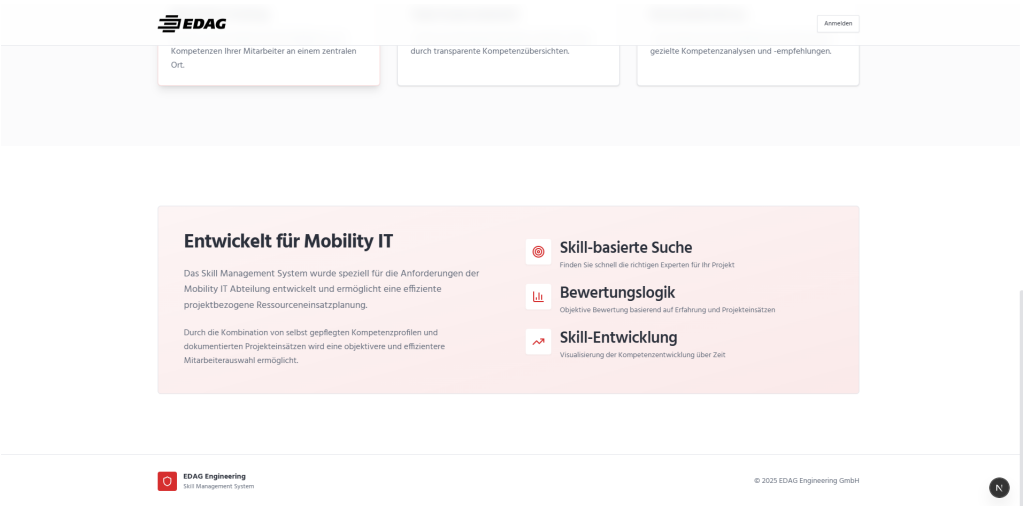


Abbildung A.20: Welcome-Seite: Skill-Auswahl

A.6.1.2 Dashboard und Home

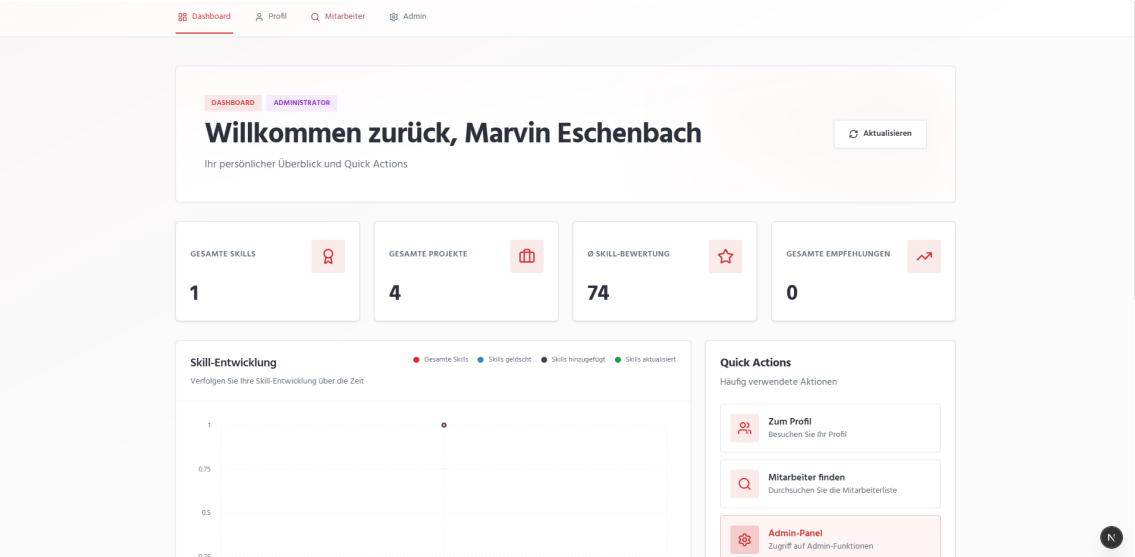


Abbildung A.21: Dashboard-Übersicht mit personalisierten Kennzahlen

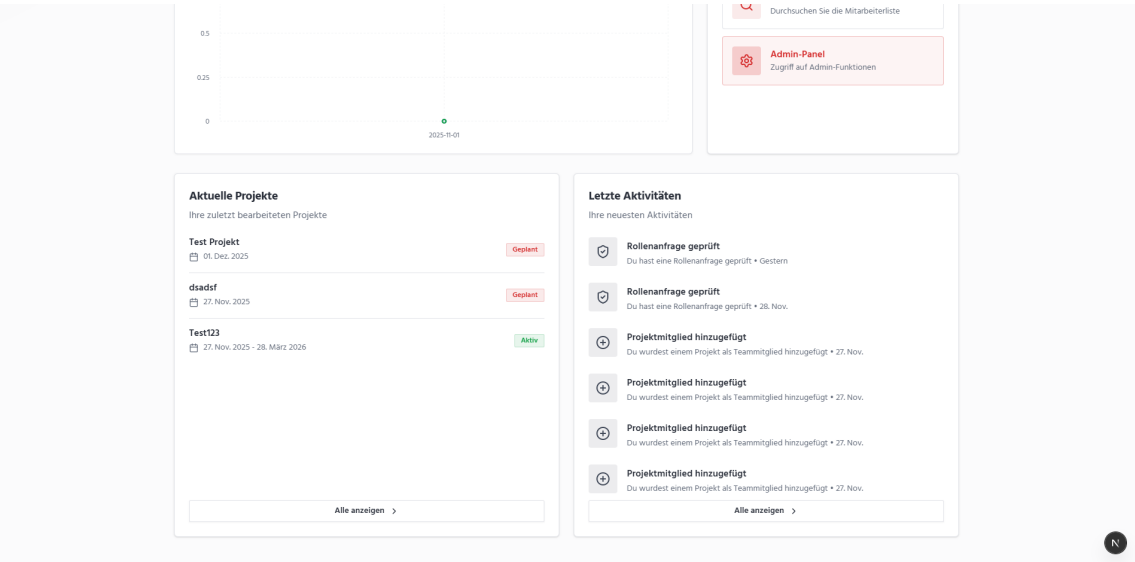


Abbildung A.22: Dashboard mit Skill-Entwicklungstrends

A.6.1.3 Profilverwaltung

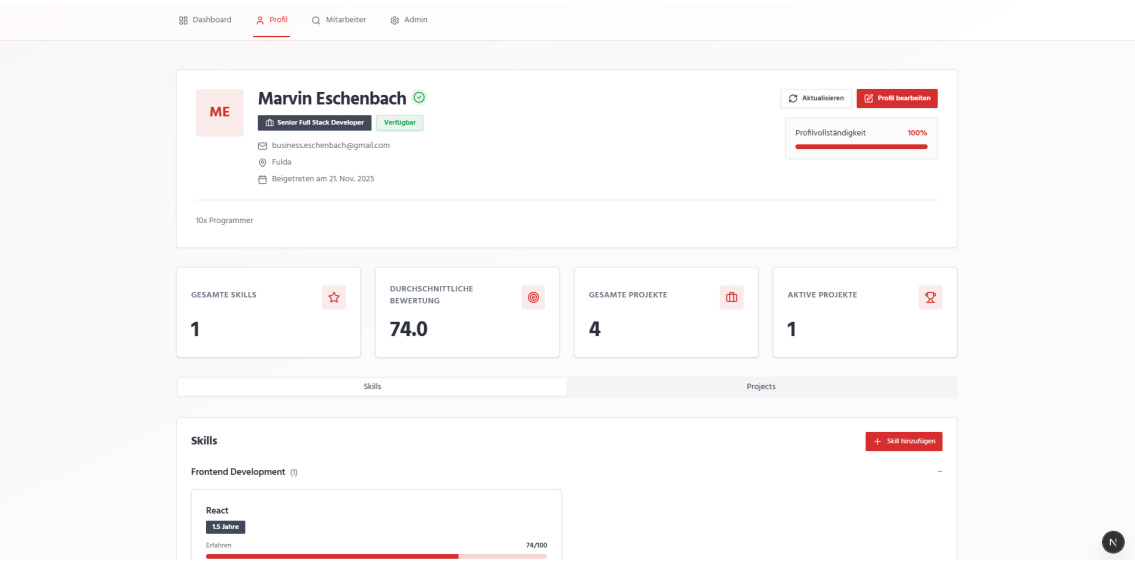


Abbildung A.23: Profil-Bearbeitung: Persönliche Daten und Skills

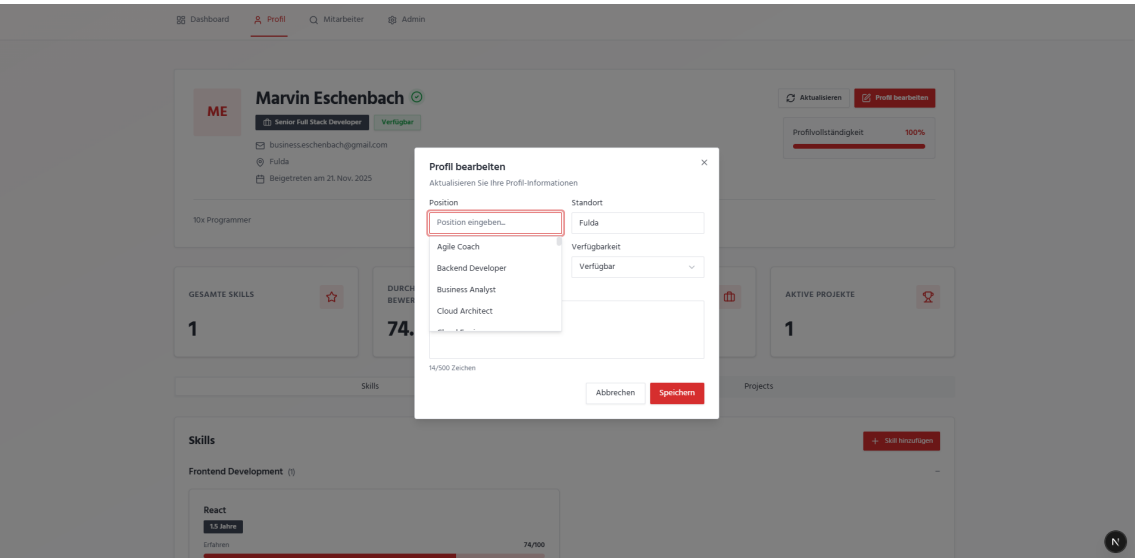


Abbildung A.24: Profil-Ansicht: Skill-Matrix mit Proficiency-Scores

A.6.1.4 Mitarbeiter- und Projektsuche

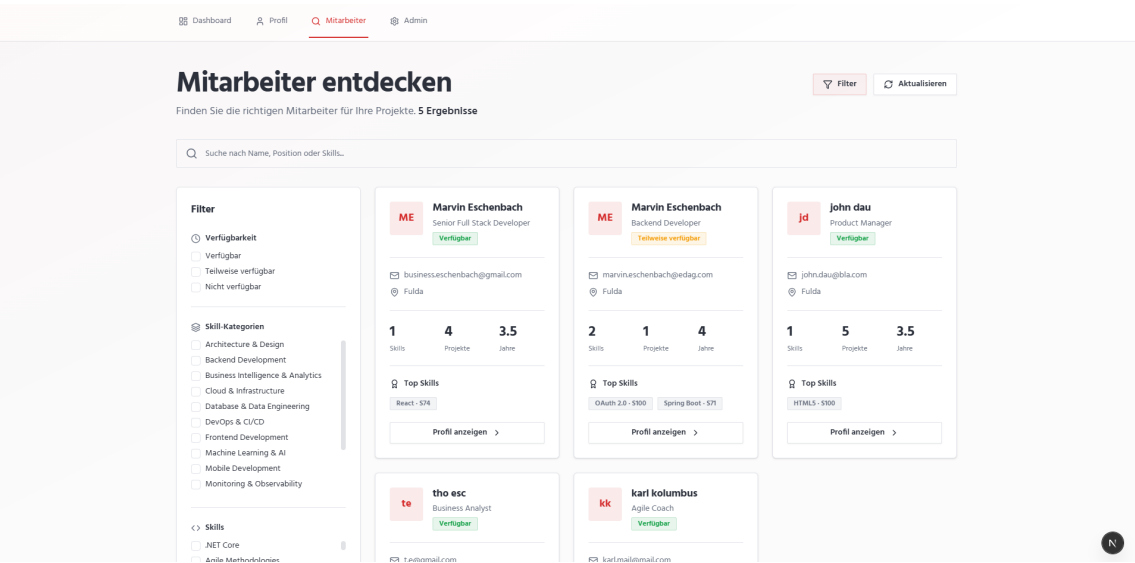


Abbildung A.25: Mitarbeitersuche mit Skill-basierten Filtern

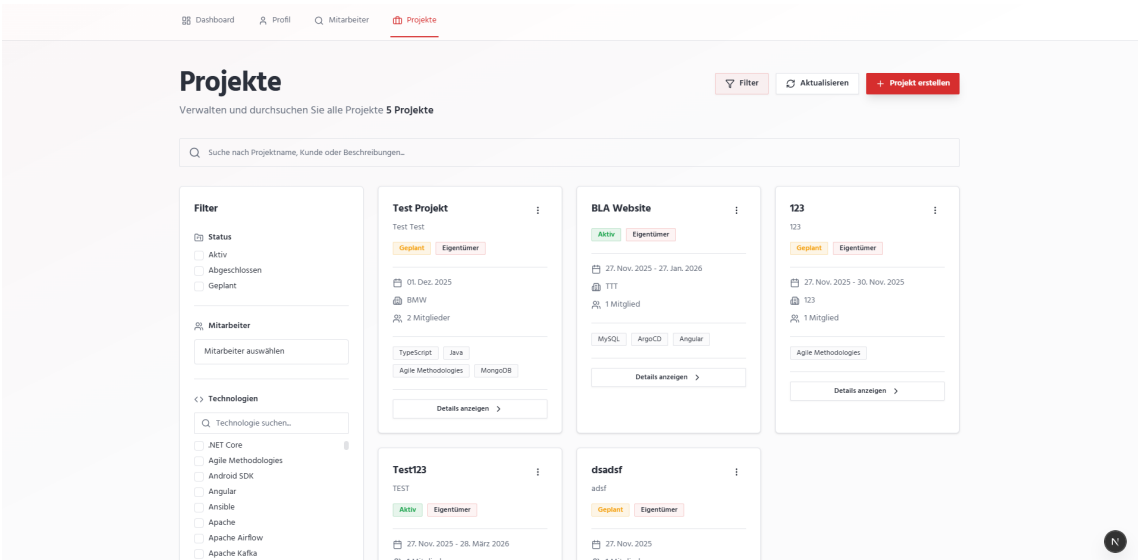


Abbildung A.26: Projektübersicht mit Status-Filter

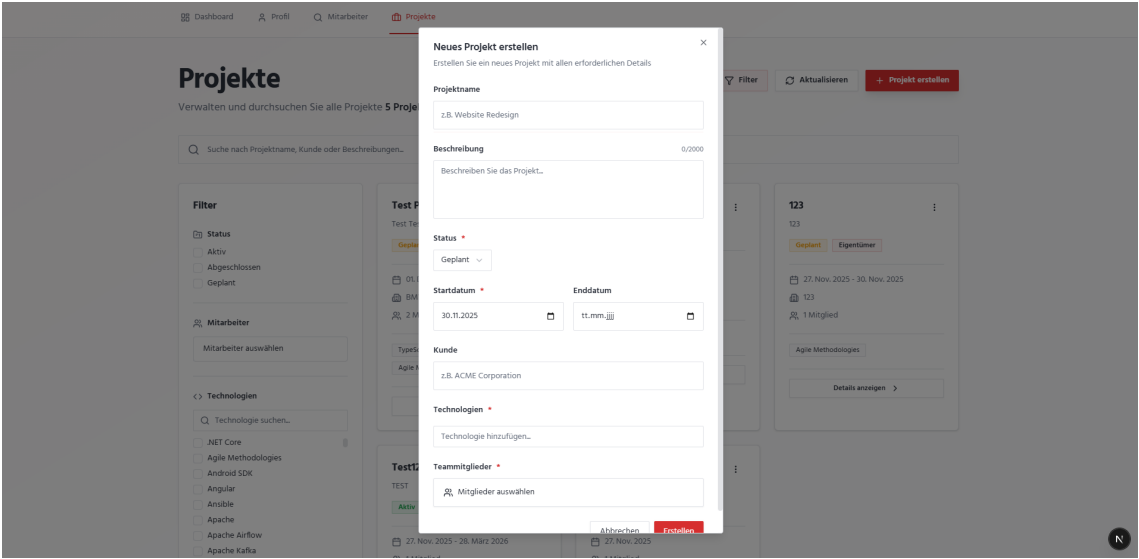


Abbildung A.27: Projekt-Detailansicht mit Team-Zusammensetzung

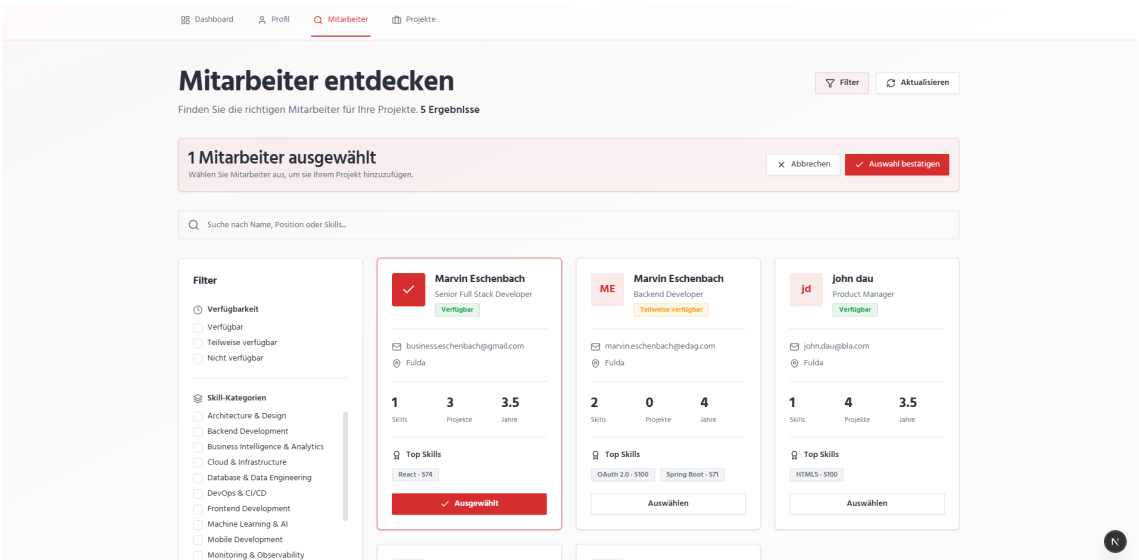


Abbildung A.28: Projekt-Erstellung: Formular für neue Projekte

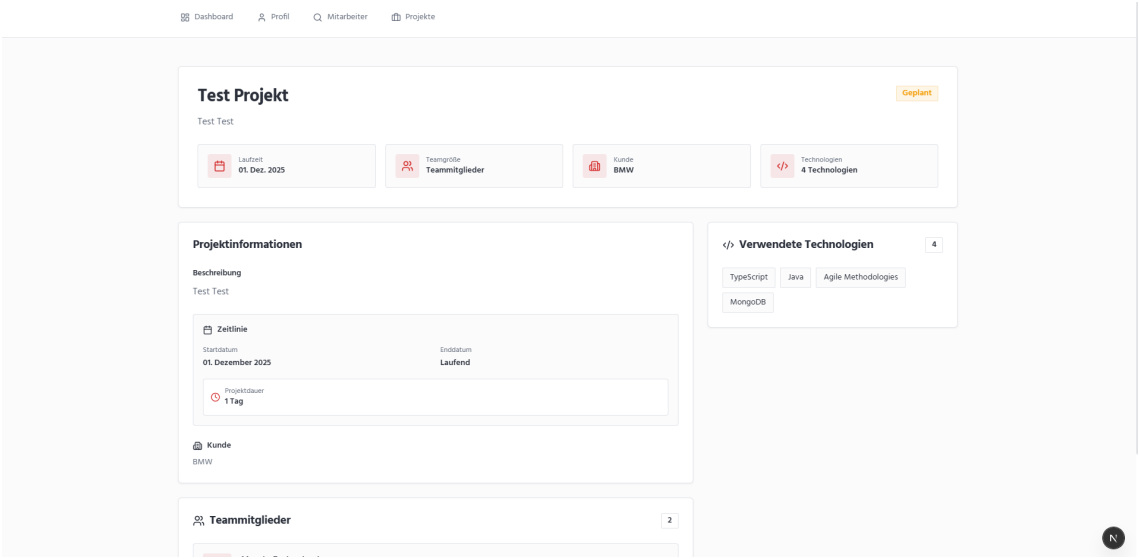


Abbildung A.29: Projekt-Team-Management: Mitglieder hinzufügen

A.6.1.5 Administrationsoberfläche

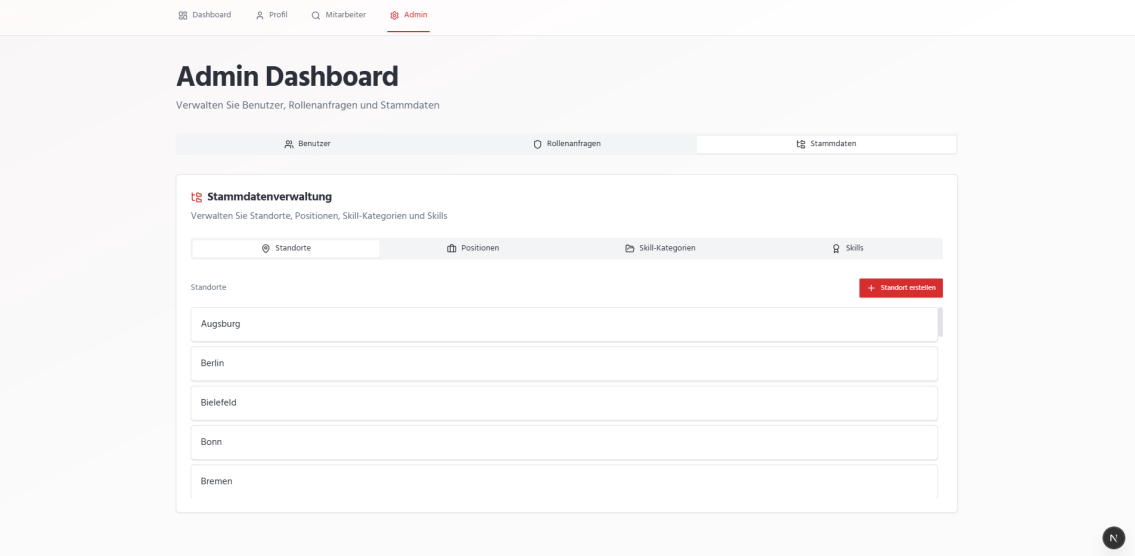


Abbildung A.30: Admin-Dashboard: Systemstatistiken und Benutzeraktivitäten

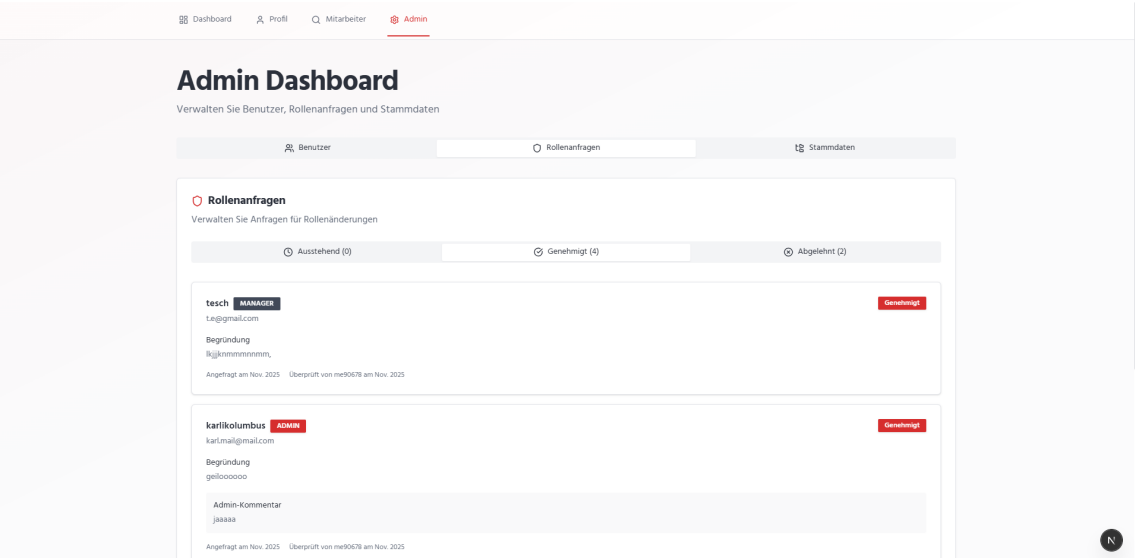


Abbildung A.31: Admin-Panel: Rollenfragen-Verwaltung

A.6.1.6 UI-Mockup (Planungsphase)

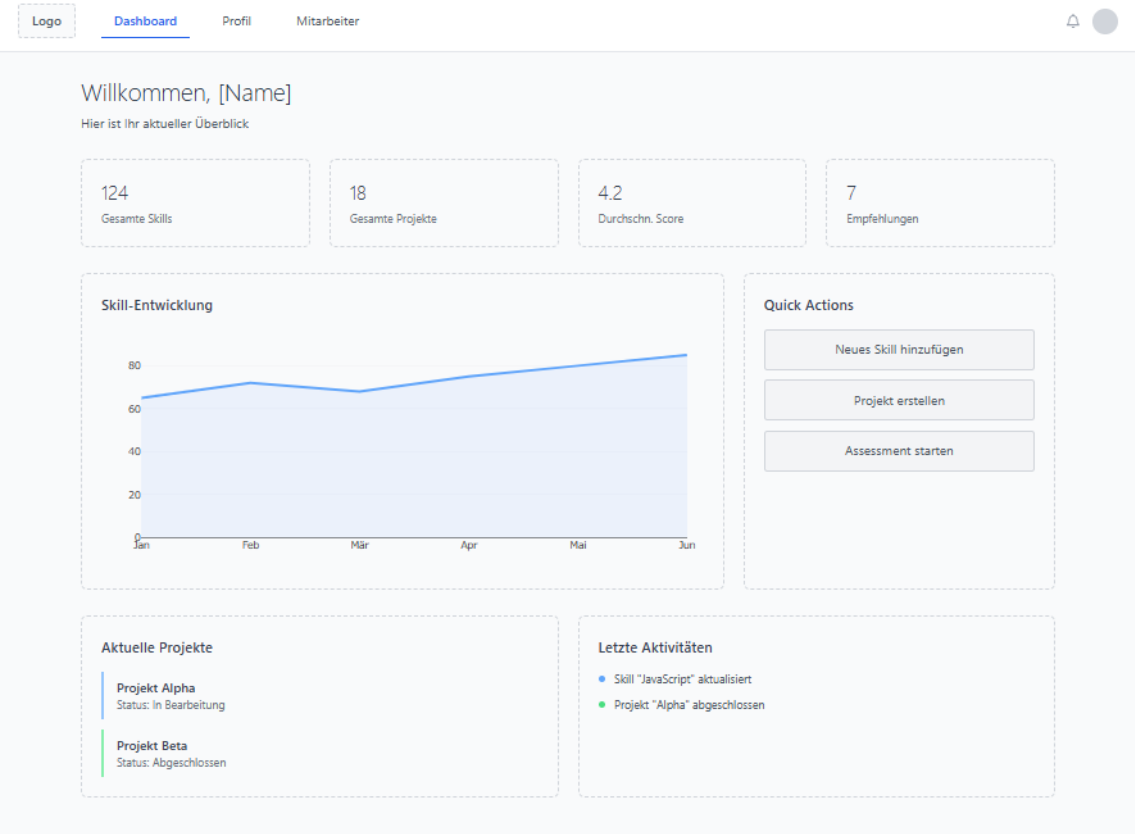


Abbildung A.32: Initiales UI-Mockup aus der Planungsphase

A.6.2 Use Cases

A.6.2.1 Übersicht der 15 implementierten Use Cases

Nr.	Use Case	Rolle
UC-01	Login über Keycloak OAuth2/OIDC	Alle
UC-02	Eigenes Profil anzeigen und bearbeiten	USER
UC-03	Skills hinzufügen/bearbeiten mit Proficiency Score	USER
UC-04	Dashboard mit Skill-Entwicklung anzeigen	USER
UC-05	Activity Feed anzeigen	USER
UC-06	Mitarbeiter suchen (Multi-Filter)	USER
UC-07	Projekte suchen	USER
UC-08	Projekt anlegen mit Mitarbeiterauswahl	MANAGER
UC-09	Projekt bearbeiten/Teammitglieder ändern	MANAGER
UC-10	Projektdetails anzeigen	MANAGER
UC-11	Rollenanfrage stellen (USER→MANAGER)	USER
UC-12	Rollenanfrage genehmigen/ablehnen	ADMIN
UC-13	Benutzerverwaltung (Rollenzuweisung)	ADMIN
UC-14	Stammdaten verwalten (Skills, Kategorien)	ADMIN
UC-15	Standorte verwalten	ADMIN

Tabelle A.9: Übersicht der 15 implementierten Use Cases

A.6.3 Benutzer- und Entwicklerdokumentation

Dokumentationsstruktur

Die Dokumentation ist in zwei Hauptbereiche unterteilt:

1. **Benutzerdokumentation** – beschreibt die Nutzung der Anwendung
2. **Entwicklerdokumentation** – beschreibt Konzept, Architektur, Umsetzung und Tests

Benutzerdokumentation

Kapitel	Inhalt
01_Einführung.md	Überblick über das System und die Zielsetzung
02_Benutzerrollen.md	Beschreibung der Rollen und Berechtigungen (Mitarbeiter, Projektleiter, Admin)
03_Funktionale_Übersicht.md	Überblick über die Hauptfunktionen (Skillverwaltung, Suche, Bewertungen, Visualisierung)
04_Benutzeranleitung.md	Schritt-für-Schritt-Anleitung zur Nutzung der Anwendung
05_FAQ_und_Support.md	Häufige Fragen, Kontakt und Hilfestellung

Entwicklerdokumentation

Die Struktur orientiert sich an den zentralen Ideen des **arc42-Modells**, jedoch vereinfacht für eine praxisgerechte Projektdokumentation im Ausbildungsrahmen.

Kapitel	Inhalt
01_Projektkontext.md	Ausgangssituation, Zielsetzung, Nutzen
02_Anforderungen.md	Funktionale & nicht-funktionale Anforderungen
03_Systemarchitektur.md	Überblick über die Architektur (Frontend, Backend, Datenbank, CI/CD)
04_Bausteine.md	Beschreibung der Hauptkomponenten und ihrer Schnittstellen
05_Datenmodell.md	Entitäten, Beziehungen und Datenflüsse
06_Technologien_und_Tools.md	Verwendete Frameworks, Libraries, Tools und Entwicklungsumgebung
07_Umsetzung.md	Implementierung der Kernfunktionen (Auth, Skills, Bewertung, Filter, Visualisierung)
08_Ergebnisse_und_Fazit.md	Bewertung des Projekts, Lessons Learned, Ausblick

Abbildung A.33: Benutzer- und Entwicklerdokumentation: Deckblatt der Benutzerdokumentation

3.2 Dashboard

Startseite nach Login

Anzeige:

- **Willkommen-Nachricht** mit Benutzername
- **Statistik-Karten:**
 - Total Skills (Anzahl hinzugefügter Skills)
 - Durchschnittlicher Proficiency Score
 - Anzahl aktiver Projekte
 - Letzte Aktivität (Datum)
- **Top Skills Chart:** Die 5 besten Skills visualisiert
- **Schnellzugriff-Buttons:** Profil bearbeiten, Rollen anfragen, Mitarbeiter suchen

3.3 Profilverwaltung

Mein Profil

Stammdaten (aus Keycloak, nicht editierbar):

- Name (Vor- und Nachname)
- E-Mail-Adresse

Editierbare Felder:

- **Position:** Dropdown (Software Engineer, Projektleiter, Teamleiter, etc.)
- **Standort:** Dropdown (Fulda, Berlin, München, Remote, etc.)
- **Biografie:** Freitext (max. 500 Zeichen) - optional
- **Erfahrung:** Zahlenfeld
- **Verfügbarkeit:** Dropdown (Verfügbar, Begrenzt verfügbar, Nicht verfügbar)

Abbildung A.34: Benutzer- und Entwicklerdokumentation: Auszug aus der funktionalen Übersicht (Kapitel 3)

Domain Services (Inbound Ports)

Service	Verantwortlichkeit
ProfileService	Profilverwaltung
SkillManagementService	Skill-Management
EmployeeDiscoveryService	Mitarbersuche
ProjectManagementService	Projektverwaltung
RoleRequestService	Rollenverwaltung
AnalyticsService	Analytics & Statistiken
OptionsService	Referenzdaten

Infrastructure Layer

REST Controllers

Controller	Endpoint	Funktionen
ProfileController	/api/profile	Profil abrufen/aktualisieren, Skills verwalten
EmployeeDiscoveryController	/api/employees	Mitarbeiter suchen
ProjectManagementController	/api/projects	Projekte erstellen/verwalten
AnalyticsController	/api/analytics	Dashboard-Daten

JPA Repositories

Optimierungen:

- EntityGraphs zur Vermeidung von N+1-Problemen
- Custom Queries mit JPQL
- Pagination Support für große Datenmengen

Security & Configuration

- OAuth2 Resource Server mit JWT-Validation
- Role-Based Access Control (RBAC)
- Method Security mit @PreAuthorize
- CORS-Konfiguration für Frontend

Abbildung A.35: Benutzer- und Entwicklerdokumentation: Auszug aus der Architekturübersicht (Kapitel 4 - Bausteine)

Die Abbildungen A.33 bis A.35 zeigen exemplarische Auszüge aus der erstellten Benutzer- und Entwicklerdokumentation, die eine umfassende Beschreibung aller Systemfunktionen, der technischen Architektur und der Deployment-Prozesse enthält.

A.6.4 API-Dokumentation

Die REST-API des Skill Management Systems folgt RESTful-Prinzipien und ist vollständig mit OpenAPI 3.0 dokumentiert. Die API umfasst 7 Hauptbereiche mit insgesamt über 30 Endpunkten für Profilverwaltung, Mitarbeitersuche, Projektmanagement, Administration und Analytics.

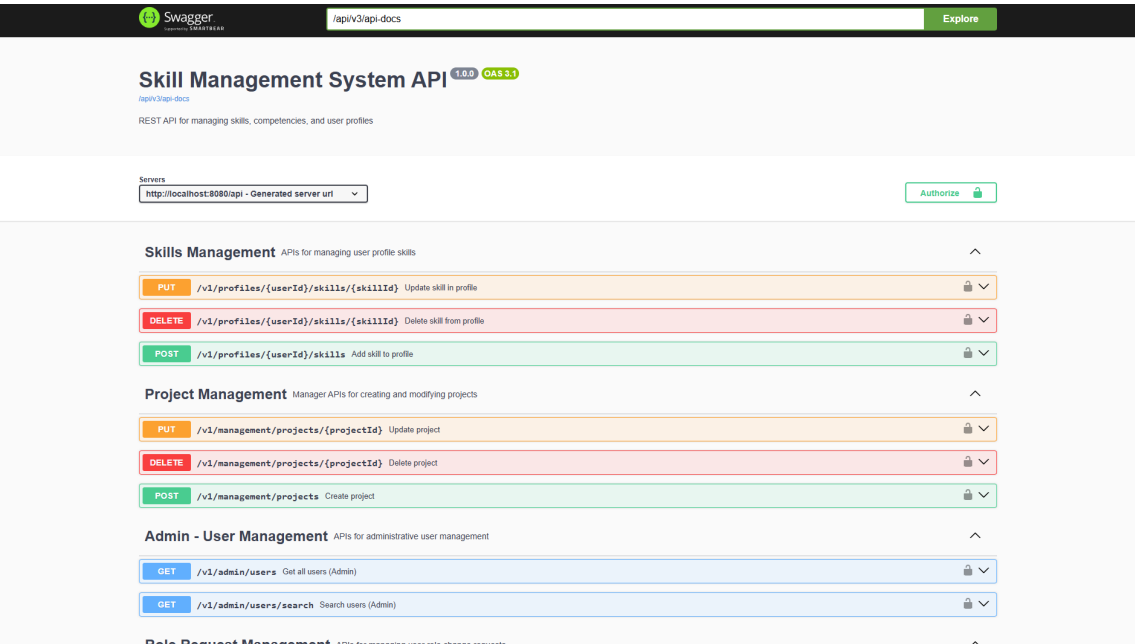


Abbildung A.36: API-Dokumentation: Übersicht der verfügbaren Endpunkte gruppiert nach Funktionsbereichen

A.6.4.1 Vollständige API-Endpunkt-Übersicht

Die folgende Tabelle listet alle implementierten API-Endpunkte mit HTTP-Methode, Pfad, Beschreibung und erforderlicher Rolle auf:

Methode	Endpunkt	Beschreibung	Rolle
Skills Management			
PUT	/v1/profiles/{userId}/skills/{skillId}	Update skill in profile	USER
DELETE	/v1/profiles/{userId}/skills/{skillId}	Delete skill from profile	USER
POST	/v1/profiles/{userId}/skills	Add skill to profile	USER
Project Management			
PUT	/v1/management/projects/{projectId}	Update project	MANAGER
DELETE	/v1/management/projects/{projectId}	Delete project	MANAGER
POST	/v1/management/projects	Create project	MANAGER
Admin - User Management			
GET	/v1/admin/users	Get all users (Admin)	ADMIN
GET	/v1/admin/users/search	Search users (Admin)	ADMIN
Role Request Management			
GET	/v1/role-requests	Get role requests by status	USER
POST	/v1/role-requests	Create role request	USER
POST	/v1/role-requests/{requestId}/reject	Reject role request	ADMIN
POST	/v1/role-requests/{requestId}/approve	Approve role request	ADMIN
GET	/v1/role-requests/me	Get my role requests	USER
DELETE	/v1/role-requests/{requestId}	Cancel role request	USER
Profile Management			
GET	/v1/profiles/{userId}	Get user profile	USER
PUT	/v1/profiles/{userId}	Update user profile	USER
Employee Discovery			
GET	/v1/employees/{userId}	Get employee by ID	USER
GET	/v1/employees/search	Search employees	USER
GET	/v1/employees/filter-options	Get filter options	USER
Projects			
GET	/v1/projects/{projectId}	Get project by ID	USER
GET	/v1/projects/users/{userId}	Get user projects	USER
GET	/v1/projects/search	Search manager's projects	MANAGER
GET	/v1/projects/filter-options	Get project filter options	USER
Analytics & Activities			
GET	/v1/analytics/users/{userId}/statistics	Get user statistics	USER
GET	/v1/analytics/users/{userId}/skill-trends	Get skill development trends	USER
GET	/v1/analytics/users/{userId}/activities	Get recent activities	USER
Reference Data Options			
GET	/v1/options/skills	Get all available skills	USER
POST	/v1/options/skills	Create skill (Admin)	ADMIN
GET	/v1/options/skill-categories	Get skill categories	USER
POST	/v1/options/skill-categories	Create skill category (Admin)	ADMIN
GET	/v1/options/positions	Get available positions	USER
POST	/v1/options/positions	Create position (Admin)	ADMIN
GET	/v1/options/locations	Get available locations	USER
POST	/v1/options/locations	Create location (Admin)	ADMIN
GET	/v1/options/skills/category/{categoryId}	Get available skills by category	USER

Tabelle A.10: Vollständige REST-API-Endpunkt-Übersicht des Skill Management Systems

Authentifizierung: Alle Endpunkte erfordern einen gültigen JWT Access Token im Authorization-Header (**Bearer <token>**). Die Token-Validierung erfolgt via Keycloak Token Introspection (siehe Sequenzdiagramm in Abbildung A.1).

Literaturverzeichnis

- [1] Eigene Erhebung. *Zeitaufwand für eine Projektbesetzung*. Durchschnittlich 2,5 Stunden basierend auf Interviews mit drei Projektbeauftragten am 03.10.2025.
- [2] Vercel Inc. *Next.js 15 Documentation*. Abgerufen am 21.10.2025 um 11:30 Uhr. URL: <https://nextjs.org/docs>.
- [3] Meta Platforms, Inc. *React – The library for web and native user interfaces*. Abgerufen am 21.10.2025 um 11:45 Uhr. URL: <https://react.dev>.
- [4] VMware, Inc. *Spring Boot 3.4 Reference Documentation*. Abgerufen am 22.10.2025 um 08:15 Uhr. URL: <https://docs.spring.io/spring-boot/index.html>.
- [5] PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. Abgerufen am 21.10.2025 um 13:00 Uhr. URL: <https://www.postgresql.org/docs/16>.
- [6] Red Hat, Inc. *Keycloak 26 Server Administration Guide*. Abgerufen am 23.10.2025 um 10:00 Uhr. URL: https://www.keycloak.org/docs/latest/server_admin.
- [7] Cloud Native Computing Foundation. *Kubernetes Documentation*. Abgerufen am 03.11.2025 um 15:30 Uhr. URL: <https://kubernetes.io/docs/home>.
- [8] Jenkins Project. *Jenkins User Documentation*. Abgerufen am 05.11.2025 um 09:00 Uhr. URL: <https://www.jenkins.io/doc>.
- [9] SonarSource SA. *SonarQube 10.x Documentation*. Abgerufen am 05.11.2025 um 14:45 Uhr. URL: <https://docs.sonarsource.com/sonarqube/latest>.
- [10] International Organization for Standardization. *ISO 9241-110:2020 – Ergonomics of human-system interaction – Part 110: Interaction principles*. Abgerufen am 09.10.2025 um 13:45 Uhr. URL: <https://www.iso.org/standard/75258.html>.
- [11] EDAG Engineering GmbH. *Das Unternehmen – Die EDAG Group im Überblick*. Abgerufen am 02.10.2025 um 10:15 Uhr. URL: <https://www.edag.com/de/edag-group/das-unternehmen>.
- [12] boerse.de. *EQS-News: EDAG Group – Positives adjusted EBIT im dritten Quartal mit weiterhin zurückhaltender Nachfrage nach Engineering-Dienstleistungen*. Abgerufen am 02.10.2025 um 10:30 Uhr. URL: <https://www.boerse.de/nachrichten/EQS-News-EDAG-Group-Positives-adjusted-EBIT-im-dritten-Quartal-mit-weiterhin-zurueckhaltender-Nachfrage-nach-Engineering-Dienstleistungen/37992264>.

- [13] Docker, Inc. *Docker Documentation*. Abgerufen am 20.10.2025 um 14:20 Uhr. URL: <https://docs.docker.com>.
- [14] Kent Beck u. a. *Manifesto for Agile Software Development*. Abgerufen am 07.10.2025 um 15:45 Uhr. URL: <https://agilemanifesto.org>.
- [15] Leah Brown. *What is DevOps? An Updated Definition for the Next Generation*. Abgerufen am 08.10.2025 um 14:00 Uhr. URL: <https://itrevolution.com/articles/what-is-devops-an-updated-definition-for-the-next-generation>.
- [16] Eigene Berechnung. *Stundensatz eines Projektleiters auf Basis branchenüblicher Durchschnittswerte*. Herleitung basierend auf veröffentlichten Durchschnittsgehältern und üblichen Gemeinkostenzuschlägen; angenommener Stundensatz: 75 Euro (Stand: 14.10.2025).
- [17] W3C. *Web Content Accessibility Guidelines (WCAG) 2.1*. Techn. Ber. Abgerufen am 07.10.2025 um 16:00 Uhr. World Wide Web Consortium. URL: <https://www.w3.org/TR/WCAG21/>.
- [18] Google LLC. *Web Performance Best Practices*. Abgerufen am 28.10.2025 um 16:15 Uhr. URL: <https://web.dev/explore/learn-core-web-vitals>.
- [19] D. Hardt. *The OAuth 2.0 Authorization Framework*. RFC 6749. Abgerufen am 08.10.2025 um 11:15 Uhr. IETF. URL: <https://datatracker.ietf.org/doc/html/rfc6749>.
- [20] Robert C. Martin. *Clean Architecture: Summary and Principles*. Abgerufen am 07.10.2025 um 09:30 Uhr. URL: <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>.
- [21] Google Cloud. *API Design Guide*. Abgerufen am 09.10.2025 um 11:00 Uhr. URL: <https://cloud.google.com/apis/design>.
- [22] Object Management Group. *Unified Modeling Language (UML) Version 2.5.1*. Abgerufen am 08.10.2025 um 13:30 Uhr. URL: <https://www.omg.org/spec/UML/2.5.1>.
- [23] Dai Clegg und Richard Barker. *Case Method Fast-Track: A RAD Approach*. MoSCoW-Priorisierung: Must have, Should have, Could have, Won't have.
- [24] Atlassian Corporation Plc. *Bitbucket Documentation*. Abgerufen am 20.10.2025 um 09:45 Uhr. URL: <https://support.atlassian.com/bitbucket>.
- [25] Christian Henning. *Interner Verrechnungssatz für Auszubildende der EDAG Engineering GmbH*. Persönliche Mitteilung vom 14.10.2025; Stundensatz: 35 Euro.
- [26] Christian Henning. *Interner Verrechnungssatz für Projektbetreuer der EDAG Engineering GmbH*. Persönliche Mitteilung vom 14.10.2025; Stundensatz: 75 Euro.
- [27] Agyleos. *Die beste Skills Management Software 2024 – Ein Vergleich*. Abgerufen am 03.10.2025 um 14:30 Uhr. URL: <https://agyleos.com/de/post/beste-skills-management-software>.
- [28] Europäische Union. *Datenschutz-Grundverordnung (DSGVO) – Verordnung (EU) 2016/679*. Abgerufen am 04.10.2025 um 11:00 Uhr. URL: <https://eur-lex.europa.eu/legal-content/DE/TXT/?uri=CELEX:32016R0679>.

- [29] SUSE LLC. *Rancher Desktop Documentation*. Abgerufen am 04.11.2025 um 10:30 Uhr. URL: <https://docs.rancherdesktop.io>.
- [30] JetBrains s.r.o. *IntelliJ IDEA Documentation*. Abgerufen am 20.10.2025 um 08:45 Uhr. URL: <https://www.jetbrains.com/help/idea>.
- [31] JetBrains s.r.o. *WebStorm Documentation*. Abgerufen am 21.10.2025 um 09:00 Uhr. URL: <https://www.jetbrains.com/help/webstorm>.
- [32] GitHub, Inc. *GitHub Copilot – Your AI pair programmer*. Abgerufen am 20.10.2025 um 16:00 Uhr. URL: <https://github.com/features/copilot>.
- [33] Conventional Commits Contributors. *Conventional Commits 1.0.0 – A specification for adding human and machine readable meaning to commit messages*. Abgerufen am 15.10.2025 um 10:00 Uhr. URL: <https://www.conventionalcommits.org/>.
- [34] Microsoft. *Database normalization description*. Abgerufen am 11.10.2025 um 14:15 Uhr. URL: <https://learn.microsoft.com/en-us/office/troubleshoot/access/database-normalization-description>.
- [35] Redgate Software Ltd. *Flyway Documentation – Database Migrations Made Easy*. Abgerufen am 22.10.2025 um 10:00 Uhr. URL: <https://flywaydb.org/documentation>.
- [36] Alistair Cockburn. *Hexagonal Architecture*. Abgerufen am 08.10.2025 um 10:30 Uhr. URL: <https://alistair.cockburn.us/hexagonal-architecture>.
- [37] Google Cloud. *REST API Design Guide*. Abgerufen am 10.10.2025 um 09:45 Uhr. URL: <https://cloud.google.com/apis/design>.
- [38] OpenAPI Initiative. *OpenAPI Specification 3.1.0*. Abgerufen am 10.10.2025 um 16:30 Uhr. URL: <https://spec.openapis.org/oas/latest.html>.
- [39] M. Jones, J. Bradley und N. Sakimura. *JSON Web Token (JWT)*. RFC. Abgerufen am 09.10.2025 um 15:00 Uhr. URL: <https://datatracker.ietf.org/doc/html/rfc7519>.
- [40] VMware, Inc. *Spring Security OAuth 2.0 Resource Server*. Abgerufen am 23.10.2025 um 14:30 Uhr. URL: <https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/index.html>.
- [41] Microsoft Corporation. *TypeScript Documentation*. Abgerufen am 21.10.2025 um 10:15 Uhr. URL: <https://www.typescriptlang.org/docs>.
- [42] Tailwind Labs Inc. *Tailwind CSS Documentation*. Abgerufen am 25.10.2025 um 11:30 Uhr. URL: <https://tailwindcss.com/docs>.
- [43] NextAuth.js Contributors. *NextAuth.js – Authentication for Next.js*. Abgerufen am 23.10.2025 um 09:00 Uhr. URL: <https://next-auth.js.org>.
- [44] Redux Team. *Redux Toolkit Documentation*. Abgerufen am 22.10.2025 um 09:30 Uhr. URL: <https://redux-toolkit.js.org>.
- [45] shadcn. *shadcn/ui – Beautifully designed components built with Radix UI and Tailwind CSS*. Abgerufen am 21.10.2025 um 12:00 Uhr. URL: <https://ui.shadcn.com>.

- [46] Recharts Contributors. *Recharts – A composable charting library built on React components*. Abgerufen am 26.10.2025 um 14:00 Uhr. URL: <https://recharts.org/en-US>.
- [47] Ilya Khorikov. *Feature-Sliced Design: Architectural methodology for frontend projects*. Abgerufen am 12.10.2025 um 10:00 Uhr. URL: <https://feature-sliced.design/>.
- [48] Robert C. Martin. *Clean Code Principles*. Abgerufen am 06.10.2025 um 15:30 Uhr. URL: <https://www.freecodecamp.org/news/clean-coding-for-beginners/>.
- [49] Google LLC. *Lighthouse – Automated auditing, performance metrics, and best practices for the web*. Abgerufen am 06.11.2025 um 11:30 Uhr. URL: <https://developer.chrome.com/docs/lighthouse/overview>.
- [50] VMware, Inc. *Spring Data JPA – Reference Documentation*. Abgerufen am 23.10.2025 um 11:00 Uhr. URL: <https://docs.spring.io/spring-data/jpa/reference>.
- [51] Docker, Inc. *Multi-stage builds*. Abgerufen am 25.10.2025 um 15:45 Uhr. URL: <https://docs.docker.com/build/building/multi-stage>.
- [52] Redis Ltd. *Redis Documentation*. Abgerufen am 27.10.2025 um 10:45 Uhr. URL: <https://redis.io/docs>.
- [53] Microsoft Corporation. *Playwright – Fast and reliable end-to-end testing*. Abgerufen am 30.10.2025 um 09:15 Uhr. URL: <https://playwright.dev/docs/intro>.
- [54] Cloud Native Computing Foundation. *Prometheus Documentation*. Abgerufen am 08.11.2025 um 10:00 Uhr. URL: <https://prometheus.io/docs>.
- [55] Grafana Labs. *Grafana Documentation*. Abgerufen am 09.11.2025 um 14:45 Uhr. URL: <https://grafana.com/docs>.
- [56] Cloud Native Computing Foundation. *Jaeger – Distributed Tracing Platform*. Abgerufen am 10.11.2025 um 11:15 Uhr. URL: <https://www.jaegertracing.io/docs>.
- [57] PostgreSQL Global Development Group. *PostgreSQL Backup and Restore*. Abgerufen am 11.11.2025 um 15:30 Uhr. URL: <https://www.postgresql.org/docs/current/backup.html>.